COMSATS University, Islamabad Pakistan

Aassignment-02

CLO-2

PodCraft – Agentic AI Podcast Episode Planning System
Version 1.0

Submitted By:

Asna Hammad FA23-BDS-051

Awais Ashraf FA23-BDS-053

Supervisor

Dr. Tehseen Riaz Abbasi

Bachelor of Science in Data Science / Software Engineering

(2023-2027)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University, Islamabad Pakistan

PodCraft – Agentic AI Podcast Episode Planning System

A project presented to

COMSATS University, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelors of Science in Data Science / Software Engineering

(2023-2027)

By

Asna Hammad FA23-BDS-051

Awais Ashraf FA23-BDS-053

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Asna Hammad

Awais Ashraf

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (DS) PodCraft was developed by Muhammad **Awais Ashraf (FA23-BDS-053)** and **Asna Hammad (FA23-BDS-051)** under the supervision of **Dr. Tehseen Riaz Abbasi** and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Data Science.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

In the rapidly growing domain of digital content creation, podcasters face significant challenges during pre-production, including time-consuming manual research, unorganized episode structuring, and difficulty in generating engaging questions and hooks. Appointing manual methods for this task involves many problems such as increased preparation time, inconsistency in content quality, lack of structured workflows, no intelligent assistance, and reliance on scattered digital tools. Talking about current planning methods, they use general note-taking applications or basic script editors irrespective of the fact that podcasters need automated research and structured outlines.

To prevent all these problems from prevailing, PodCraft is developed. It is an agentic AI-based podcast planning system, which provides solutions to many of these problems. It is a cross-platform application (web and mobile) which allows podcasters to input their niche or guest information. It monitors different content requirements using intelligent autonomous agents which can research topics based on user input and generate structured episode plans for later reference as well.

PodCraft is a podcast planning system that can be assigned to a content type, as in our case podcast episodes, it works accordingly, analyzes user intent upon receiving niche or guest details, and performs user-defined actions like generating questions, hooks, timestamps, and complete episode outlines for future use.

It is an intelligent system consisting of a React JS web frontend, React Native mobile app, Node JS backend with Express, and MongoDB database. OpenAI API is used for AI agent processing, and Firebase is used for cloud authentication and real-time synchronization. LAN and internet communication are used by users to access the platform and operate the system manually or through automated agents.

Abstract

PodCraft is developed to simplify and automate podcast episode planning using intelligent autonomous agents.
Currently, podcasters rely heavily on manual research, scattered online sources, and unstructured planning methods, which consume significant time and often reduce content quality.
Many creators struggle to organize episode flow, prepare engaging questions, and design strong introductions that retain audience attention. This leads to inconsistent production quality and inefficient preparation workflows.
There is no unified intelligent platform that autonomously plans podcast episodes based on niche and guest information.
PodCraft addresses this gap by using autonomous agents to research topics, generate structured questions, create engaging hooks, and design complete episode outlines. The system aims to automate preparation, improve episode structure, and assist creators in producing professional content efficiently. It also enhances time management by providing timestamped segments and guided content flow.
The project ultimately enables podcasters to focus more on creativity and delivery rather than planning logistics. PodCraft improves productivity and raises the overall quality of podcast production through intelligent automation.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor Dr. Tehseen Riaz Abbasi. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are grateful to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us with the values of honesty & hard work.

Asna Hammad

Awais Ashraf

Table of Contents

Abstract 5

Chapter 5: Testing and Evaluation 8

1.1 Web Dashboard Testing: 8
1.1.1 Unit Testing: 8
1.1.2 Functional Testing 36
1.1.3 Business Rules Testing 43
1.1.4 Integration Testing 51
1.2 Mobile App: 59
1.2.1 Unit Testing 59
1.2.2 Functional Testing 67
1.2.3 Business Rules Testing 69
1.2.4 Integration Testing 72
1.1 5.5 Summary of Results 75

Chapter 5: Testing and Evaluation

Once the system has been successfully developed, testing has to be conducted to ensure that the system works as intended. This is also to check that the system meets the requirements stated earlier. Besides that, system testing will help in finding the errors that may be hidden from the user. The testing must be completed before it is deployed for use.
There are few types of testing which include unit testing, functional testing and integration testing.
For **PodCraft**, testing has been performed on **Modules 1, 2, and 3** which include:

- **Module 1:** User Authentication & Profile Management
- **Module 2:** User Intent & Guest Analyzer
- **Module 3:** Topic Research Agent

1.1 Web Dashboard Testing:

1.1.1 Unit Testing:

Unit testing verifies the smallest testable components of the software (e.g., individual functions, methods, or classes) in isolation. The purpose is to ensure that each unit performs as expected, independent of the full system.
For PodCraft, unit tests were written for core backend utility functions using the Detailed Test Case Format as per lecture guidelines.

1.1.1.1 MODULE 1: User Authentication & Profile Management:

1.1.1.1.1 Unit Test Case 1.1: validateEmail() Function:

Test case ID:	UT_1_1_001
Test case description:	To verify that email validation correctly identifies valid and invalid email formats
Preconditions:	Email validation function is imported
Test steps:	1. Call with valid email 2. Call with missing @ 3. Call with missing domain 4. Call with spaces 5. Call with empty string 6. Call with special chars only
Test data:	1. "user@example.com" 2. “user@example.com" 3. "user@" 4. "user @example.com " 5. "" 6. "@#$%^&*()"
Expected Result:	1. True 2. False 3. False

	4. False 5. False 6. False
Actual Result:	1. True 2. False 3. False 4. False 5. False 6. False
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-09
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.2 Unit Test Case 1.2: validatePasswordStrength() Function

Test case ID:	UT_1_2_001
Test case description:	To verify password strength validation (min 8 chars, uppercase, lowercase, number, special char)
Preconditions:	Password validation function is imported
Test steps:	1. Call with strong password 2. Call with short password 3. Call with no uppercase 4. Call with no lowercase 5. Call with no number 6. Call with no special char 7. Call with empty string 8. Call with only spaces
Test data:	1. "PodCraft@2025" 2. "Pod@1" 3. "podcraft@2025" 4. "PODCRAFT@2025" 5. "PodCraft@" 6. "PodCraft2025" 7. "" 8. " "
Expected Result:	1. True 2. False (min length) 3. False (uppercase missing) 4. False (lowercase missing) 5. False (number missing) 6. False (special char missing) 7. False (empty) 8. False (only spaces)

Actual Result:	1. True 2. False (min length) 3. False (uppercase missing) 4. False (lowercase missing) 5. False (number missing) 6. False (special char missing) 7. False (empty) 8. False (only spaces)
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-09
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.3 Unit Test Case 1.3: generateJWT() Function:

Test case ID:	UT_1_3_001
Test case description:	To verify JWT token generation with correct payload and expiration
Preconditions:	JWT library configured with secret key
Test steps:	1. Generate token with valid userId 2. Decode token and verify userId 3. Verify expiration time (24 hours) 4. Generate with null userId 5. Generate with empty string userId 6. Verify token has 3 parts
Test data:	1. userId = "user_12345" 2. Token from step 1 3. Token from step 1 4. userId = null 5. userId = "" 6. Token from step 1
Expected Result:	1. Valid JWT string 2. userId matches "user_12345" 3. exp = iat + 86400 4. Error 5. Error 6. True (3 parts separated by dots)
Actual Result:	1. "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." (valid token) 2. userId: "user_12345" 3. exp = iat + 86400 4. Error: "Invalid user ID" 5. Error: "User ID cannot be empty" 6. FAIL: Token has only 2 parts (missing signature) when userId contains special characters

Test Case Status:	Fail
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.4 Unit Test Case 1.4: verifyJWT() Function

Test case ID:	UT_1_4_001
Test case description:	To verify JWT token verification works for valid, expired, and tampered tokens
Preconditions:	Valid JWT token exists from UT_1_3_001
Test steps:	1. Verify valid token 2. Verify expired token (simulate by modifying exp) 3. Verify tampered token (change one character) 4. Verify empty token 5. Verify malformed token (missing part)
Test data:	1. Valid token from UT_1_3_001 2. Token with expired exp claim 3. Valid token with last char changed 4. "" 5. "abc.def" (only 2 parts)
Expected Result:	1. Returns decoded payload 2. Error: "Token expired" 3. Error: "Invalid token" 4. Error: "No token provided" 5. Error: "Malformed token"
Actual Result:	1. Returns decoded payload 2. Error: "Token expired" 3. Error: "Invalid token" 4. Error: "No token provided" 5. Error: "Malformed token"
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.5 Unit Test Case 1.5: hashPassword() Function

Test case ID:	UT_1_5_001
Test case description:	To verify password hashing produces bcrypt hash with correct format
Preconditions:	bcrypt library installed
Test steps:	1. Hash valid password 2. Verify hash starts with 2b2*b*

	3. Hash same password twice, verify different salts 4. Hash empty password 5. Hash very long password (100 chars)
Test data:	1. "SecurePass@123" 2. Hash from step 1 3. "SecurePass@123" twice 4. "" 5. 100-character string
Expected Result:	1. Returns string 2. Starts with "2b2*b*" 3. Two different hash strings 4. Error 5. Returns valid hash
Actual Result:	1. abcdefghijklmnopqrstuvwxyz1234567890" 2.Startswith"2b$" 3. Two different hashes generated 4. Error: "Password cannot be empty" 5. FAIL: Very long password (100 chars) causes bcrypt to throw "Invalid salt revision" error
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.6 Unit Test Case 1.6: comparePassword() Function

Test case ID:	UT_1_6_001
Test case description:	To verify password comparison works correctly for matching and non-matching passwords
Preconditions:	Hash from UT_1_5_001 is available
Test steps:	1. Compare correct password with hash 2. Compare incorrect password with hash 3. Compare empty password with hash 4. Compare password with wrong hash format 5. Compare null values
Test data:	1. Plain: "SecurePass@123", Hash: from UT_1_5_001 2. Plain: "WrongPass@123", Same hash 3. Plain: "", Same hash 4. Plain: "SecurePass@123", Hash: "invalidhash" 5. Plain: null, Hash: null
Expected Result:	1. True 2. False 3. False 4. False 5. Error

Actual Result:	1. True 2. False 3. False 4. False 5. Error: "Invalid input parameters"
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.7 Unit Test Case 1.7: validateNicheCategory() Function

Test case ID:	UT_1_7_001
Test case description:	To verify niche validation accepts only predefined taxonomy categories
Preconditions:	Predefined niche list: ["Technology", "Business", "Health", "Education", "True Crime", "Comedy", "Sports", "News", "Arts", "Science"]
Test steps:	1. Call with valid niche "Technology" 2. Call with valid niche "True Crime" 3. Call with invalid niche "Random" 4. Call with empty string 5. Call with null 6. Call with case-sensitive mismatch "technology" (lowercase)
Test data:	1. "Technology" 2. "True Crime" 3. "Random Stuff" 4. "" 5. null 6. "technology"
Expected Result:	1. True 2. True 3. False 4. False 5. False 6. False (case-sensitive) or True (if case-insensitive - document behavior)
Actual Result:	1. True 2. True 3. False 4. False 5. False 6. False (case-sensitive validation works)
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15

Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.8 Unit Test Case 1.8: validateEpisodeDuration() Function

Test case ID:	UT_1_8_001
Test case description:	To verify episode duration validation (between 5 and 180 minutes)
Preconditions:	Duration validation function is available
Test steps:	1. Call with 30 minutes 2. Call with 5 minutes (minimum) 3. Call with 180 minutes (maximum) 4. Call with 4 minutes (below minimum) 5. Call with 181 minutes (above maximum) 6. Call with negative number 7. Call with zero 8. Call with non-number input
Test data:	1. 30 2. 5 3. 180 4. 4 5. 181 6. -10 7. 0 8. "abc"
Expected Result:	1. True 2. True 3. True 4. False (min 5 minutes) 5. False (max 180 minutes) 6. False 7. False 8. False
Actual Result:	1. True 2. True 3. True 4. False - "Duration must be at least 5 minutes" 5. False - "Duration cannot exceed 180 minutes" 6. False - "Duration cannot be negative" 7. False - "Duration must be at least 5 minutes" 8. False - "Duration must be a number"
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.9 Unit Test Case 1.9: checkUserEpisodeLimit() Function (Free vs Premium)

Test case ID:	UT_1_9_001
Test case description:	To verify episode limit checking based on user tier (Free: 3/month, Premium: unlimited)
Preconditions:	Function receives user tier and current month episode count
Test steps:	1. Free user with 0 episodes 2. Free user with 2 episodes 3. Free user with 3 episodes 4. Free user with 3 episodes but new month 5. Premium user with 3 episodes 6. Premium user with 100 episodes 7. Invalid tier
Test data:	1. Tier: "free", count: 0 2. Tier: "free", count: 2 3. Tier: "free", count: 3 4. Tier: "free", count: 3, new month: true 5. Tier: "premium", count: 3 6. Tier: "premium", count: 100 7. Tier: "unknown", count: 0
Expected Result:	1. True (allowed) 2. True (allowed) 3. False (limit reached) 4. True (month reset) 5. True (unlimited) 6. True (unlimited) 7. False (invalid tier)
Actual Result:	1. True 2. True 3. False - "Monthly limit of 3 episodes reached" 4. True 5. True 6. FAIL: Premium user with 100 episodes returns False (incorrectly blocking due to integer overflow in database)
Test Case Status:	Fail
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.10 Unit Test Case 1.10: sanitizeInput() Function (Security)

Test case ID:	UT_1_10_001
Test case description:	To verify input sanitization removes XSS and SQL injection attempts
Preconditions:	Sanitization function is available
Test steps:	1. Sanitize normal text 2. Sanitize HTML script tag

	3. Sanitize SQL injection attempt 4. Sanitize mixed malicious content 5. Sanitize empty string 6. Sanitize very long string
Test data:	1. "Hello World" 2. "<script>alert('XSS')</script>" 3. "' OR '1'='1" 4. "<img src=x onerror=alert(1)> DROP TABLE users;" 5. "" 6. "A" repeated 10000 times
Expected Result:	1. "Hello World" 2. Escaped or removed script tags 3. Escaped quotes 4. Cleaned string 5. "" 6. Truncated or safely handled
Actual Result:	1. "Hello World" 2. "alert('XSS')" (script tags removed) 3. "' OR '1'='1" (quotes escaped) 4. "DROP TABLE users" (image tag removed) 5. "" 6. Truncated to 5000 characters
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.11 Unit Test Case 1.11: validateUserProfileData() Function

Test case ID:	UT_1_11_001
Test case description:	To verify user profile data validation (name, display name, bio)
Preconditions:	Profile validation function is available
Test steps:	1. Valid profile data 2. Name too short (<2 chars) 3. Name too long (>50 chars) 4. Bio too long (>500 chars) 5. Display name with special chars 6. Empty display name (use email as fallback)
Test data:	1. { name: "John Podcaster", displayName: "JohnP", bio: "Tech podcaster" } 2. { name: "J", displayName: "JohnP" } 3. { name: "A".repeat(51), displayName: "JohnP" } 4. { name: "John", bio: "B".repeat(501) }

	5. { displayName: "<script>alert(1)</script>" } 6. { name: "John", displayName: "" }
Expected Result:	1. True, validated profile 2. False - "Name too short" 3. False - "Name too long" 4. False - "Bio too long" 5. False after sanitization 6. True, fallback to email
Actual Result:	1. True 2. False - "Name must be at least 2 characters" 3. False - "Name cannot exceed 50 characters" 4. False - "Bio cannot exceed 500 characters" 5. False - "Display name contains invalid characters" 6. True - displayName set to email prefix
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.12 Unit Test Case 1.12: generatePasswordResetToken() Function

Test case ID:	UT_1_12_001
Test case description:	To verify password reset token generation with expiration
Preconditions:	Token generation function is available
Test steps:	1. Generate token for valid email 2. Verify token is unique 3. Verify token expiration (1 hour) 4. Generate for non-existent email 5. Verify token format 6. Verify token can be validated
Test data:	1. "john@example.com" 2. Same email twice 3. Token from step 1 4. "nonexistent@example.com" 5. Token from step 1 6. Token from step 1
Expected Result:	1. Returns unique token string 2. Two different tokens 3. exp = now + 3600 seconds 4. Returns error or null 5. 32+ character alphanumeric 6. validateToken() returns true
Actual Result:	1. "a7f3e9d2c4b8a1f6e3d9c2b7a4f8e1d6" 2. Two different tokens generated ✓ 3. exp = now + 3600 ✓

	4. Error: "Email not found" 5. 32 characters, alphanumeric ✓ 6. validateToken() returns true
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.13 Unit Test Case 1.13: validateUserRole() Function

Test case ID:	UT_1_13_001
Test case description:	To verify user role validation (free, premium, admin)
Preconditions:	Role validation function is available
Test steps:	1. Validate "free" role 2. Validate "premium" role 3. Validate "admin" role 4. Validate invalid role "guest" 5. Validate empty role 6. Validate null role 7. Case sensitivity test
Test data:	1. "free" 2. "premium" 3. "admin" 4. "guest" 5. "" 6. null 7. "FREE"
Expected Result:	1. True 2. True 3. True 4. False 5. False 6. False 7. False (case-sensitive)
Actual Result:	1. True 2. True 3. True 4. False 5. False 6. False 7. False - "FREE" not recognized
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad

Date of execution:	2025-05-15

1.1.1.1.14 Unit Test Case 1.14: checkGuestLimitPerEpisode() Function

Test case ID:	UT_1_14_001
Test case description:	To verify guest limit validation (max 3 guests per episode per BR-3)
Preconditions:	Guest limit function is available
Test steps:	1. Add 1st guest (allowed) 2. Add 2nd guest (allowed) 3. Add 3rd guest (allowed) 4. Add 4th guest (blocked) 5. Remove a guest, then add new guest 6. Check limit for premium user
Test data:	1. Episode has 0 guests 2. Episode has 1 guest 3. Episode has 2 guests 4. Episode has 3 guests 5. Episode has 3 guests, remove 1, add new 6. Premium user with custom limit
Expected Result:	1. True 2. True 3. True 4. False - "Max 3 guests" 5. True 6. True (if premium allows more)
Actual Result:	1. True 2. True 3. True 4. False - "Maximum 3 guests per episode" 5. True - guest added after removal 6. True - premium allows up to 5 guests
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.1.15 Unit Test Case 1.15: logUserActivity() Function

Test case ID:	UT_1_15_001
Test case description:	To verify user activity logging (login, episode creation, research, export)
Preconditions:	Activity log database table exists
Test steps:	1. Log login activity 2. Log episode creation 3. Log research generation 4. Log export action

	5. Verify timestamp is recorded 6. Verify user_id is recorded correctly
Test data:	1. userId="user_123", action="login" 2. userId="user_123", action="episode_created", episodeId="ep_001" 3. userId="user_123", action="research_generated" 4. userId="user_123", action="export_pdf" 5. Any log entry 6. Log entries for user_123
Expected Result:	1. Log entry created 2. Log entry with episodeId 3. Log entry created 4. Log entry created 5. Timestamp present 6. All entries have userId="user_123"
Actual Result:	1. Success - log entry ID 1001 2. Success - log entry ID 1002 with episodeId 3. Success - log entry ID 1003 4. Success - log entry ID 1004 5. Timestamp: 2025-05-12 14:30:22 ✓ 6. All 4 entries have correct userId ✓
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2 MODULE 2: User Intent & Guest Analyzer:

1.1.1.2.1 Unit Test Case 2.1: analyzeUserIntent() Function

Test case ID:	UT_2_1_001
Test case description:	To verify user intent analysis correctly identifies episode direction based on niche and input
Preconditions:	NLP/AI function is available
Test steps:	1. Analyze intent with niche and topic 2. Analyze intent with niche and guest name 3. Analyze intent with only niche (no topic) 4. Analyze intent with empty niche 5. Analyze intent with ambiguous input 6. Analyze intent with very long input
Test data:	1. { niche: "Technology", topic: "Future of AI" } 2. { niche: "Business", guest: "Elon Musk" } 3. { niche: "Health", topic: "" } 4. { niche: "", topic: "Meditation" } 5. { niche: "Comedy", topic: "???" } 6. 1000 character string

Expected Result:	1. Returns intent with direction = "topic-focused" 2. Returns intent with direction = "guest-focused" 3. Returns "niche-exploration" direction 4. Error 5. Returns best-guess intent 6. Handles without crashing
Actual Result:	1. { direction: "topic-focused", confidence: 0.92 } 2. { direction: "guest-focused", confidence: 0.88 } 3. { direction: "niche-exploration", confidence: 0.65 } 4. Error: "Niche is required" 5. FAIL: { direction: null, confidence: 0.0 } (ambiguous input causes crash instead of graceful fallback) 6. FAIL: 1000 character input causes API timeout (no truncation implemented)
Test Case Status:	Fail
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2.2 Unit Test Case 2.2: extractGuestInfoFromURL() Function

Test case ID:	UT_2_2_001
Test case description:	To verify guest information extraction from Wikipedia, LinkedIn, or personal website URLs
Preconditions:	Web crawling function with network access
Test steps:	1. Extract from valid Wikipedia URL 2. Extract from valid LinkedIn URL 3. Extract from invalid URL 4. Extract from inaccessible URL (404) 5. Extract from empty URL 6. Extract from non-English URL
Test data:	1. "https://en.wikipedia.org/wiki/Elon_Musk" 2. "https://linkedin.com/in/satya" 3. "https://not-a-real-website.com" 4. "https://example.com/404-page" 5. "" 6. "https://fr.wikipedia.org/wiki/Marie_Curie"
Expected Result:	1. Returns name, bio, key achievements 2. Returns name, headline, summary 3. Error or fallback to manual entry 4. Error 5. Error 6. Returns extracted info
Actual Result:	1. { name: "Elon Musk", bio: "CEO of Tesla and SpaceX...", achievements: ["Tesla", "SpaceX"] }

	2. { name: "Satya Nadella", headline: "CEO of Microsoft", summary: "..." } 3. Error: "Unable to reach URL - falling back to manual entry" 4. Error: "404 - Page not found" 5. Error: "URL cannot be empty" 6. { name: "Marie Curie", bio: "Physicist and chemist..." }
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2.3 Unit Test Case 2.3: generateGuestSummary() Function

Test case ID:	UT_2_3_001
Test case description:	To verify AI generates coherent guest summary from extracted information
Preconditions:	OpenAI API is configured
Test steps:	1. Generate summary for valid guest data 2. Generate with minimal data (only name) 3. Generate with detailed data 4. Generate with empty data 5. Verify summary length (100-300 words) 6. Generate for same guest twice
Test data:	1. { name: "Elon Musk", achievements: ["Tesla", "SpaceX"], profession: "CEO" } 2. { name: "Jane Doe" } 3. Full profile with bio, achievements, links 4. { } 5. Summary from test 1 6. Same data as test 1
Expected Result:	1. 150-250 word summary 2. Shorter summary requesting more info 3. 250-300 word detailed summary 4. Error 5. Within range 6. Similar but not identical
Actual Result:	1. 187 word summary generated ✓ 2. "Jane Doe - additional information would help create a more detailed summary" 3. 278 word detailed summary ✓ 4. Error: "Guest data is required" 5. 187 words ✓ 6. Similar content, different phrasing (acceptable)
Test Case Status:	Pass
Created By:	Asna Hammad

Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2.4 Unit Test Case 2.4: determineEpisodeTone() Function

Test case ID:	UT_2_4_001
Test case description:	To verify episode tone determination based on niche and user preference
Preconditions:	Tone determination function is available
Test steps:	1. Determine tone for Comedy niche 2. Determine tone for Business niche 3. Determine tone for True Crime niche 4. Override with user preference 5. Determine tone with no niche 6. Determine tone for mixed signals
Test data:	1. { niche: "Comedy", userPreference: null } 2. { niche: "Business", userPreference: null } 3. { niche: "True Crime", userPreference: null } 4. { niche: "Technology", userPreference: "Humorous" } 5. { niche: "", userPreference: null } 6. { niche: "News", userPreference: "Casual" }
Expected Result:	1. "Humorous" 2. "Professional" 3. "Serious" 4. "Humorous" 5. Default "Casual" or error 6. "Casual"
Actual Result:	1. "Humorous" 2. "Professional" 3. "Serious" 4. "Humorous" (user preference applied) 5. Error: "Niche required for tone determination" 6. "Casual"
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2.5 Unit Test Case 2.5: extractKeywordsFromIntent() Function

Test case ID:	UT_2_5_001
Test case description:	To verify keyword extraction from user intent for downstream agents
Preconditions:	NLP keyword extraction is available
Test steps:	1. Extract from simple topic 2. Extract from complex topic

	3. Extract from guest name only 4. Extract from niche + topic 5. Extract from foreign language input 6. Extract from very short input
Test data:	1. "AI ethics" 2. "The impact of quantum computing on modern cryptography" 3. "Interview with Marie Curie" 4. { niche: "Health", topic: "mental wellness" } 5. "Inteligencia artificial en español" 6. "AI"
Expected Result:	1. ["AI", "ethics"] 2. ["quantum computing", "cryptography"] 3. ["Marie Curie"] 4. ["Health", "mental wellness"] 5. Handles gracefully 6. ["AI"]
Actual Result:	1. ["AI", "ethics"] 2. ["quantum computing", "cryptography", "impact"] 3. ["Marie Curie"] 4. ["Health", "mental wellness"] 5. ["inteligencia", "artificial", "español"] (Spanish supported) 6. ["AI"]
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2.6 Unit Test Case 2.6: savePodcastProfile() Function

Test case ID:	UT_2_6_001
Test case description:	To verify podcast profile saving with correct data structure
Preconditions:	Database available, user authenticated
Test steps:	1. Save new profile 2. Save profile with duplicate name 3. Update existing profile 4. Delete profile 5. Save profile for free user (max 2) 6. Save profile for premium user (max 10)
Test data:	1. { name: "TechPod", niche: "Technology", tone: "Professional" } 2. Same name "TechPod" 3. Updated to "TechPod Pro" 4. Existing profile ID 5. Free user with 2 existing profiles 6. Premium user with 10 existing profiles

Expected Result:	1. Saved, returns profileId 2. Error: "Profile name exists" 3. Updated successfully 4. Deleted 5. Error: "Max 2 profiles for free tier" 6. Error: "Max 10 profiles"
Actual Result:	1. profileId = "prof_001" 2. Error: "Profile name already exists" 3. Success - name changed to "TechPod Pro" 4. Success - profile removed 5. Error: "Free tier limited to 2 profiles. Upgrade to premium." 6. Error: "Maximum 10 profiles reached"
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2.7 Unit Test Case 2.7: detectContentDirection() Function

Test case ID:	UT_2_7_001
Test case description:	To verify content direction detection (educational, entertaining, inspirational, news, debate)
Preconditions:	Content detection function is available
Test steps:	1. Detect for "How to" topic 2. Detect for "Top 10" topic 3. Detect for "Why X is wrong" topic 4. Detect for "Interview with expert" 5. Detect for "Latest news" topic 6. Detect for ambiguous topic
Test data:	1. "How to start a podcast" 2. "Top 10 AI tools" 3. "Why cryptocurrency is a scam" 4. { guest: "Expert", type: "interview" } 5. "Breaking news in tech today" 6. "AI"
Expected Result:	1. "educational" 2. "entertaining" 3. "debate" 4. "educational" 5. "news" 6. Best-guess direction
Actual Result:	1. "educational" 2. "entertaining" 3. "debate" 4. "educational"

	5. "news" 6. "informational" (default guess)
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2.8 Unit Test Case 2.8: validateIntentDataIntegrity() Function

Test case ID:	UT_2_8_001
Test case description:	To verify intent data integrity checks (required fields, data types, foreign keys)
Preconditions:	Intent validation function is available
Test steps:	1. Validate complete intent record 2. Validate missing niche 3. Validate missing topic AND guest 4. Validate invalid data types 5. Validate user_id foreign key existence 6. Validate duration within bounds
Test data:	1. { userId: "user_123", niche: "Tech", topic: "AI", duration: 45 } 2. { userId: "user_123", topic: "AI" } 3. { userId: "user_123", niche: "Tech" } 4. { userId: 123, niche: "Tech", duration: "forty five" } 5. { userId: "invalid", niche: "Tech", topic: "AI" } 6. { userId: "user_123", niche: "Tech", duration: 200 }
Expected Result:	1. True 2. False (niche required) 3. False (topic/guest required) 4. False 5. False 6. False (duration >180)
Actual Result:	1. True ✓ 2. False - "Niche is required" 3. False - "Either topic or guest is required" 4. False - "Invalid data type for duration" 5. False - "User ID does not exist" 6. False - "Duration cannot exceed 180 minutes"
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2.9 Unit Test Case 2.9: detectGuestSentiment() Function

Test case ID:	UT_2_9_001

Test case description:	To verify guest sentiment analysis from bio/social media (positive, neutral, controversial)
Preconditions:	Sentiment analysis API is available
Test steps:	1. Analyze positive guest bio 2. Analyze neutral guest bio 3. Analyze controversial guest 4. Analyze empty bio 5. Analyze very short bio 6. Analyze non-English bio
Test data:	1. "Award-winning humanitarian and philanthropist" 2. "Software engineer with 10 years experience" 3. "Controversial political commentator known for bold statements" 4. "" 5. "Doctor" 6. "Médecin et chercheur français"
Expected Result:	1. "positive" 2. "neutral" 3. "controversial" or "mixed" 4. "neutral" (default) 5. "neutral" 6. Sentiment detected
Actual Result:	1. "positive" (confidence: 0.89) 2. "neutral" (confidence: 0.76) 3. "controversial" (confidence: 0.82) 4. "neutral" (default) ✓ 5. "neutral" (confidence: 0.60) 6. "positive" (confidence: 0.72) - French supported
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.2.10 Unit Test Case 2.10: generateSuggestedTopicsFromIntent() Function

Test case ID:	UT_2_10_001
Test case description:	To verify AI generates suggested topics based on user intent (before full research)
Preconditions:	OpenAI API is configured
Test steps:	1. Generate suggestions for Technology niche 2. Generate for Health niche 3. Generate for guest name 4. Generate with empty intent 5. Verify 3-5 suggestions 6. Verify suggestions are relevant

Test data:	1. { niche: "Technology", topic: "AI" } 2. { niche: "Health", topic: "Mental wellness" } 3. { guest: "Elon Musk" } 4. { } 5. Suggestions from test 1 6. Suggestions from test 1
Expected Result:	1. 3-5 AI-related topics 2. 3-5 health-related topics 3. 3-5 Musk-related topics 4. Error 5. Count = 3-5 6. Highly relevant to input
Actual Result:	1. ["GPT-5 Release", "AI in Healthcare", "Ethics of AI", "AI Job Displacement"] (4 topics) 2. ["Mindfulness Techniques", "Therapy Apps", "Stress Management", "Work-Life Balance"] (4 topics) 3. ["Tesla Updates", "SpaceX Mars Mission", "Neuralink Progress", "X.AI Launch"] (4 topics) 4. Error: "Intent data required" 5. 4 topics generated ✓ 6. All highly relevant ✓
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3 MODULE 3: Topic Research Agent

1.1.1.3.1 Unit Test Case 3.1: searchTopicsByNiche() Function

Test case ID:	UT_3_1_001
Test case description:	To verify topic search returns relevant topics based on niche only
Preconditions:	Search API (Perplexity/Bing) is configured
Test steps:	1. Search for "Technology" niche 2. Search for "Health" niche 3. Search for "True Crime" niche 4. Search with empty niche 5. Search with very specific niche 6. Search with invalid niche
Test data:	1. "Technology" 2. "Health and Wellness" 3. "True Crime" 4. ""

	5. "Quantum computing in healthcare" 6. "NicheThatDoesNotExist"
Expected Result:	1. Array of 10-15 topics 2. Health-related topics 3. Crime-related topics 4. Error or empty array 5. Specific relevant topics 6. Empty or fallback topics
Actual Result:	1. 12 topics returned (AI, Cybersecurity, Cloud, etc.) 2. 11 topics returned (Nutrition, Exercise, Mental Health) 3. 10 topics returned (Murder cases, Unsolved mysteries) 4. Error: "Niche is required" 5. FAIL: Very specific niche "Quantum computing in healthcare" returns 0 topics but no fallback to broader search6. 0 topics, message: "No topics found. Try a different niche."
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3.2 Unit Test Case 3.2: searchTopicsByGuest() Function

Test case ID:	UT_3_2_001
Test case description:	To verify topic search returns topics related to a specific guest
Preconditions:	Search API is configured
Test steps:	1. Search for "Elon Musk" 2. Search for "Taylor Swift" 3. Search for unknown person 4. Search with empty guest name 5. Search for guest with common name 6. Search for guest with special characters
Test data:	1. "Elon Musk" 2. "Taylor Swift" 3. "NonExistentPerson123" 4. "" 5. "Mike" 6. "José García"
Expected Result:	1. Topics about Tesla, SpaceX 2. Topics about music, Eras Tour 3. Few or no results 4. Error 5. Ambiguous results 6. Relevant results
Actual Result:	1. ["Tesla Cybertruck", "SpaceX Starship", "Neuralink", “X.AI"] ✓ 2. ["Eras Tour", "New Album", "Taylor Swift Merch"] ✓

	3. 0 results, fallback to niche search 4. Error: "Guest name is required" 5. 5 results (multiple "Mike" personas - mixed relevance) 6. ["José García Biography", "José García Filmography"] ✓
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3.3 Unit Test Case 3.3: filterTopicsByRelevance() Function

Test case ID:	UT_3_3_001
Test case description:	To verify topic filtering by relevance score (threshold 0.65)
Preconditions:	Topic list with relevance scores is available
Test steps:	1. Filter with default threshold (0.65) 2. Filter with high threshold (0.8) 3. Filter with low threshold (0.3) 4. Filter with empty list 5. Filter topics missing scores 6. Filter with negative threshold
Test data:	1. Topics: [{score:0.9}, {score:0.7}, {score:0.6}, {score:0.5}] 2. Same, threshold=0.8 3. Same, threshold=0.3 4. [] 5. [{title:"A"}, {title:"B"}] 6. threshold=-0.5
Expected Result:	1. Returns topics with score >=0.65 (0.9,0.7) 2. Returns score >=0.8 (0.9) 3. Returns all 4. Returns [] 5. Returns all or treats as 0 6. Uses default
Actual Result:	1. 2 topics returned (0.9, 0.7) 2. 1 topic returned (0.9) 3. 4 topics returned (all) 4. [] 5. All 2 topics returned (treated as score=0.5) 6. Used default threshold 0.65
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3.4 Unit Test Case 3.4: filterTopicsByRecency() Function

Test case ID:	UT_3_4_001
Test case description:	To verify topic filtering by date (last 30 days, last 6 months, all time)
Preconditions:	Topics with dates are available
Test steps:	1. Filter "Last 30 days" 2. Filter "Last 6 months" 3. Filter "All time" 4. Filter empty list 5. Filter topics missing dates 6. Filter invalid range
Test data:	1. Topics with dates: today, 15 days ago, 45 days ago, 200 days ago 2. Same topics 3. Same topics 4. [] 5. [{title:"A"}] 6. "Last 100 years"
Expected Result:	1. Returns topics from last 30 days (today, 15 days) 2. Returns from last 6 months (today, 15 days, 45 days) 3. Returns all 4. [] 5. Returns all (no date filter) 6. Returns all or uses default
Actual Result:	1. 2 topics returned (today, 15 days) 2. 3 topics returned (today, 15 days, 45 days) 3. 4 topics returned (all) 4. [] 5. 1 topic returned (no date filtering) 6. All 4 topics returned (defaulted to "All time")
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3.5 Unit Test Case 3.5: bookmarkTopic() Function

Test case ID:	UT_3_5_001
Test case description:	To verify bookmarking functionality (add, remove, count limits)
Preconditions:	Episode with research results exists
Test steps:	1. Bookmark a topic 2. Verify bookmark flag is true 3. Bookmark same topic again (toggle) 4. Bookmark 10 topics (max) 5. Attempt to bookmark 11th 6. Unbookmark, verify count decreases

Test data:	1. Episode ID="ep_001", Topic ID="topic_001" 2. Same topic 3. Same topic 4. 10 different topics 5. 11th topic 6. One bookmarked topic
Expected Result:	1. Success, count=1 2. Flag=true 3. Removed, count=0 4. Success, count=10 5. Error: "Max 10 bookmarks" 6. Count decreases to 9
Actual Result:	1. Success - bookmark added ✓ 2. isBookmarked = true ✓ 3. Success - bookmark removed, count=0 ✓ 4. Success - 10 bookmarks added ✓ 5. Error: "Maximum 10 bookmarks per episode" ✓ 6. Count decreased to 9 ✓
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3.6 Unit Test Case 3.6: rejectTopic() Function

Test case ID:	UT_3_6_001
Test case description:	To verify topic rejection functionality (hide topic, prevent reappearance)
Preconditions:	Episode with research results exists
Test steps:	1. Reject a topic 2. Verify not shown in active list 3. Refresh research, verify not re-added 4. Reject already rejected topic 5. Unreject topic (if feature exists) 6. Verify rejected count in DB
Test data:	1. Episode ID="ep_001", Topic ID="topic_002" 2. Same topic 3. Same niche/topic combination 4. Same rejected topic again 5. Rejected topic ID 6. Episode with 3 rejected topics
Expected Result:	1. Success 2. Not visible 3. Filtered out

	4. Error or no change 5. Restored to active 6. Count=3
Actual Result:	1. Success - topic rejected 2. Topic not in active list 3. Rejected topic excluded from new results 4. Error: "Topic already rejected" 5. Success - topic restored 6. rejected_count = 3 in episode record
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3.7 Unit Test Case 3.7: cacheResearchResults() Function

Test case ID:	UT_3_7_001
Test case description:	To verify research results caching (24-hour TTL per BR-5)
Preconditions:	Redis cache is configured
Test steps:	1. Perform research, verify cached 2. Same query within 24h, verify cache hit 3. Same query after 25h, verify cache miss 4. Different query, verify new cache entry 5. Clear cache, verify miss 6. Check TTL on cached entry
Test data:	1. Niche="Tech", Topic="AI" 2. Same niche+topic 3. Same, wait 25 hours 4. Niche="Tech", Topic="Blockchain" 5. Redis flush 6. Cached entry from test 1
Expected Result:	1. Cached, TTL=86400 2. Cache hit (<100ms) 3. New API call, cache updated 4. New cache entry 5. Miss on next query 6. TTL <= 86400
Actual Result:	1. Cached - TTL set to 86400 2. Cache hit - 45ms response 3. Cache miss - API called (2800ms), new cache created 4. FAIL: Different query (Tech/Blockchain) incorrectly returns cached results from Tech/AI (cache key collision) 5. Cache miss on next query 6. TTL = 86400 seconds
Test Case Status:	Fail

Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3.8 Unit Test Case 3.8: prioritizeBookmarkedTopics() Function

Test case ID:	UT_3_8_001
Test case description:	To verify bookmarked topics are prioritized in research results display
Preconditions:	Episode has mixed bookmarked and non-bookmarked topics
Test steps:	1. Bookmark topics B1, B2, B3 2. View research results 3. Verify bookmarked appear first 4. Unbookmark B2, verify ordering updates 5. Bookmark new topic, verify moves to top 6. Filter "Show Bookmarked Only"
Test data:	1. Topics T1-T10, bookmark T1,T5,T9 2. Same episode 3. Display order 4. Unbookmark T5 5. Bookmark T2 6. Apply bookmark filter
Expected Result:	1. Bookmarks saved 2. T1,T5,T9 shown first 3. Bookmarked at top 4. T5 moves down 5. T2 moves to top 6. Only T1,T2,T9 shown
Actual Result:	1. 3 bookmarks saved 2. Order: T1, T5, T9, T2, T3, T4, T6, T7, T8, T10 3. Bookmarked topics in positions 1-3 4. New order: T1, T9, T2, T3, T4, T5, T6, T7, T8, T10 5. T2 moves to position 2 after T1 6. Only T1, T2, T9 displayed
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3.9 Unit Test Case 3.9: generateTopicSummary() Function

Test case ID:	UT_3_9_001
Test case description:	To verify AI generates concise summary for each research topic (max 3 sentences per FR3.2)
Preconditions:	OpenAI API is configured

Test steps:	1. Generate summary for simple topic 2. Generate for complex topic 3. Verify length (≤3 sentences) 4. Generate duplicate, verify consistency 5. Generate with no context 6. Verify source attribution
Test data:	1. "The rise of artificial intelligence" 2. "Quantum entanglement in black holes" 3. Summary from test 1 4. Same as test 1 5. "" 6. Topic with source URL
Expected Result:	1. 1-2 sentences 2. 2-3 sentences 3. ≤3 sentences 4. Similar summary 5. Error or placeholder 6. Mentions/links source
Actual Result:	1. "AI has grown rapidly in recent years, transforming industries from healthcare to finance." (1 sentence) 2. 3 sentences explaining quantum entanglement 3. 1 sentence - within limit 4. Similar content, slightly different phrasing 5. Error: "Topic cannot be empty" 6. "According to Source: MIT Technology Review..."
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.1.3.10 Unit Test Case 3.10: exportResearchToPDF() Function

Test case ID:	UT_3_10_001
Test case description:	To verify research results export to PDF with proper formatting
Preconditions:	PDF generation library is configured
Test steps:	1. Export research with 10 topics 2. Verify PDF contains all topics 3. Verify includes timestamps and sources 4. Export filtered results (bookmarked) 5. Export empty research 6. Verify PDF file size and format
Test data:	1. Episode with 10 topics 2. Generated PDF 3. Generated PDF 4. Filter="bookmarked", 3 topics

	5. Episode with no research 6. PDF from test 1
Expected Result:	1. PDF generated 2. 10 topics visible 3. Sources included 4. 3 topics only 5. Error or "No data" 6. <5MB, valid PDF
Actual Result:	1. "research_export_ep001.pdf" generated ✓ 2. All 10 topics visible in PDF ✓ 3. Each topic has source URL and timestamp ✓ 4. PDF contains only 3 bookmarked topics ✓ 5. Error: "No research data to export" ✓ 6. 1.2MB, valid PDF format ✓
Test Case Status:	Pass
Created By:	Asna Hammad
Date of creation:	2025-05-15
Executed By:	Asna Hammad
Date of execution:	2025-05-15

1.1.2 Functional Testing

Functional testing validates that the system modules work correctly as a whole, ensuring that the developed system meets its specifications and requirements. Unlike unit testing, which focuses on internal functions, functional testing evaluates user-facing features through the UI or APIs.

1.1.2.1 Functional Testing 1: User Registration with Different Scenarios

Objective: To ensure that user registration works correctly for valid data and shows appropriate errors for invalid data.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Register with valid credentials	Name: "John Podcaster", Email: "john@example.com", Password: "Pass@1234"	Account created, verification email sent, redirect to verification notice	Account created, email received, redirect to notice screen	Pass
2	Register with existing email	Name: "John Doe", Email: "john@example.com", Password: "Pass@1234"	Error: "An account with this email already exists"	Error: "An account with this email already exists"	Pass
3	Register with weak password	Name: "Test User", Email: "test@example.com", Password: "weak"	Error: Password must be at least 8 characters with uppercase,	Error shown: "Password must be at least 8 characters with uppercase,	Pass

			number, and special character	number, and special character"	
4	Register with invalid email format	Name: "Test User", Email: "notanemail", Password: "Pass@1234"	Error: "Please enter a valid email address"	Error: "Please enter a valid email address"	Pass
5	Register with missing name	Name: "", Email: "test@example.com", Password: "Pass@1234"	Error: "Name is required"	Error: "Name is required"	Pass
6	Register via Google OAuth	Click "Continue with Google", select Google account	Account created automatically, redirect to dashboard	Redirected to dashboard with user profile loaded	Pass

1.1.2.2 Functional Testing 2: User Login with Different Roles

Objective: To ensure that the correct dashboard with the correct navigation bar is loaded for each user role.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Login as 'Free Tier' user	Email: "john@example.com", Password: "Pass@1234" (Free tier)	Free dashboard with limited features (3 episodes/month banner)	Free dashboard loaded, banner shows "3 episodes remaining this month"	Pass
2	Login as 'Premium' user	Email: "premium@example.com", Password: "Prem@2025"	Premium dashboard with unlimited features and "Premium" badge	Premium dashboard loaded, badge displayed, no episode limit shown	Pass
3	Login as 'Admin' user	Email: "admin@podcraft.com", Password: "Admin@2025"	Admin dashboard with user management, content monitoring, and system configuration options	FAIL: Admin dashboard loads but "Content Monitoring" menu is missing	Fail
4	Login with incorrect password	Email: "john@example.com", Password: "WrongPass123"	Error: "Incorrect email or password"	Error: "Incorrect	Pass

				email or password"	
5	Login with unregistered email	Email: "nonexistent@example.com", Password: "Pass@1234"	Error: "No account found with this email"	Shows"No account found" message	Pass
6	Login after 5 failed attempts	5 wrong attempts, then 6th attempt with correct password	Account locked for 15 minutes, lockout message displayed	Account locked, message: "Too many failed attempts. Try again in 15 minutes"	Pass

1.1.2.3 Functional Testing 3: Forgot Password / Password Recovery:

Objective: To ensure users can recover their password via email reset link.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Request reset with registered email	Email: "john@example.com"	Password reset email sent within 30 seconds	Email received in 25 seconds	Pass
2	Request reset with unregistered email	Email: "unknown@example.com"	Error: "No account found with this email"	Error: "No account found with this email"	Pass
3	Request reset with empty email	Email: ""	Error: "Email is required"	Error: "Email is required"	Pass
4	Use valid reset link	Click link from email, enter new password "NewPass@1234"	Password updated, redirect to login	Password updated, login successful with new password	Pass
5	Use expired reset link	Click link after 31 minutes	Error: "Reset link has expired. Please request a new one"	FAIL: Link still works after 31 minutes (expiration not enforced)	Fail
6	Use tampered reset token	Modify one character in reset link URL	Error: "Invalid or tampered reset token"	Error: "Invalid reset token"	Pass

1.1.2.4 Functional Testing 4: Create and Manage Podcast Profile

Objective: To ensure users can create, edit, view, and delete podcast profiles.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Create new podcast profile	Profile Name: "TechTalks", Niche: "Technology", Tone: "Professional"	Profile saved, appears in profile list	Profile created, appears in list with correct details	Pass
2	Create profile with duplicate name	Profile Name: "TechTalks" (already exists)	Error: "Profile name already exists"	Error: "Profile name already exists"	Pass
3	Edit existing profile	Change niche from "Technology" to "AI & Machine Learning"	Profile updated with new niche	Profile shows updated niche	Pass
4	View profile details	Click on "TechTalks" profile	Full profile details displayed (name, niche, tone, created date)	Details displayed but **created date shows wrong timezone (UTC instead of local)**	**Fail**
5	Delete profile	Click delete on "TechTalks", confirm deletion	Profile removed from list	Profile deleted, removed from list	Pass
6	Free user saves 3rd profile	Free user already has 2 profiles, attempts to save 3rd	Error: "Maximum 2 profiles for free tier. Upgrade to premium."	Error displayed with upgrade link	Pass

1.1.2.5 Functional Testing 5: Provide Podcast Niche and Episode Topic (Module 2)

Objective: To ensure the User Intent & Guest Analyzer correctly accepts and processes podcast niche and episode topic inputs.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Select niche from dropdown and enter topic	Niche: "Business & Entrepreneurship", Topic: "The Future of Remote Work"	Intent saved, AI agent triggered, redirect to research dashboard	Intent saved, processing started, research dashboard loads	Pass
2	Enter guest name instead of topic	Niche: "Technology", Topic: "Elon Musk" (guest name)	System detects guest name, triggers guest analysis mode	Guest analysis mode activated, guest summary generated	Pass
3	Submit without selecting niche	Niche: (empty), Topic: "Leadership"	Error: "Please select a niche"	Error: "Please select a niche"	Pass

4	Submit without topic or guest	Niche: "Comedy", Topic: ""	Error: "Please enter an episode topic or guest name"	**FAIL: Shows "Please select a niche" instead of correct error**	**Fail**
5	Free user exceeds monthly limit	Free user with 3 episodes this month, attempts 4th episode	Error: "Monthly limit reached (3 episodes). Upgrade to continue."	Error displayed, upgrade prompt with pricing	Pass
6	Save podcast profile for reuse	Click "Save as Profile", name: "Leadership Series"	Profile saved to list for future use	Profile saved, appears in "Saved Profiles" dropdown	Pass

1.1.2.6 Functional Testing 6: Guest Profile Submission (Module 2)

Objective: To ensure the system correctly accepts, analyzes, and stores guest profile information.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Submit guest with Wikipedia URL	Name: "Elon Musk", URL: "https://en.wikipedia.org/wiki/Elon_Musk"	AI crawls URL, generates guest summary (name, bio, achievements)	Summary generated: "Elon Musk is CEO of Tesla and SpaceX..."	Pass
2	Submit guest with LinkedIn URL	Name: "Satya Nadella", URL: "https://linkedin.com/in/satyanadella"	AI extracts name, headline, and summary from LinkedIn	Headline extracted: "CEO at Microsoft", summary generated	Pass
3	Submit guest with invalid URL	Name: "Test Guest", URL: "https://not-a-real-website.com"	Error or fallback to manual entry with warning	Warning: "Unable to crawl URL. Please enter guest bio manually."	Pass
4	Manual guest bio entry (no URL)	Name: "Jane Doe", Bio: "AI researcher with 10 years experience at Google"	Guest profile saved without crawling, bio stored as provided	Profile saved, bio displayed correctly	Pass
5	Submit guest without name	Name: "", Bio: "Some bio content", URL: ""	Error: "Guest name is required"	Error: "Guest name is required"	Pass

6	Add 4th guest to episode	Episode already has 3 guests, attempt to add 4th	Error: "Maximum 3 guests per episode"	FAIL: Allows adding 4th guest (limit not enforced in UI)	Fail

1.1.2.7 Functional Testing 7: Initiate and View Topic Research (Module 3)

Objective: To ensure the Topic Research Agent correctly initiates research and displays results.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Auto-trigger research after intent submission	Niche: "Technology", Topic: "AI in Healthcare"	Research starts automatically, results within 30 seconds	Research started, results displayed in 25 seconds (12 topics)	Pass
2	Manual research refresh	Click "Refresh Research" button while viewing existing results	New research results replace or supplement existing	New results loaded (replaced previous), timestamp updated	Pass
3	Research with no internet connection	Disable WiFi, trigger research	Error: "Internet connection required. Please check your network."	Error banner displayed with retry button	Pass
4	Premium user research depth	Premium account, niche: "Business", topic: "Startup Funding"	Results from 5 source categories (News, Academic, Social, Blogs, Podcasts)	All 5 categories shown, each labeled with source icon	Pass
5	Free tier user research depth	Free account, niche: "Business", topic: "Startup Funding"	Results from 2 source categories (News and Blogs only)	Only News and Blogs shown, other categories hidden	Pass
6	No relevant topics found	Niche: "Quantum Banana Physics", Topic: "Fruit Quantum States"	Message: "No relevant topics found. Try a different niche or add a guest."	FAIL: Shows blank screen instead of friendly error message	Fail

1.1.2.8 Functional Testing 8: Filter, Bookmark, and Reject Topics (Module 3)

Objective: To ensure users can filter, bookmark, and reject research topics effectively.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Filter by relevance (High only)	Research results displayed, click "High relevance only"	Only topics with relevance score > 0.7 shown	7 out of 12 topics shown (scores 0.72-0.95)	Pass

2	Filter by recency (Last 30 days)	Click "Last 30 days" filter	Only topics from past 30 days shown	FAIL: Shows a topic from 32 days ago (off-by-one error)	Fail
3	Filter by recency (Last 6 months)	Click "Last 6 months" filter	Topics from past 6 months shown	9 topics shown (correct)	Pass
4	Bookmark a topic	Click bookmark icon on topic "AI Ethics"	Topic highlighted, bookmark count increases by 1	Topic highlighted in blue, count shows 1	Pass
5	View only bookmarked topics	Click "Show Bookmarked" filter	Only bookmarked topics displayed	Only 1 topic shown (AI Ethics)	Pass
6	Bookmark 10th topic	Bookmark 9 topics, then bookmark 10th	10th bookmark saved successfully	10 bookmarks saved, all highlighted	Pass
7	Bookmark 11th topic	Attempt to bookmark 11th topic	Error: "Maximum 10 bookmarks per episode. Remove one first."	Error displayed, bookmark not added	Pass
8	Reject a topic	Click "Reject" on an irrelevant topic	Topic removed from active list, moved to rejected section	Topic disappears from current view, "Undo" option appears briefly	Pass
9	Attempt question generation without bookmarks	0 bookmarks, click "Generate Questions"	Error: "Please bookmark at least 1 topic before generating questions"	Error displayed, generation blocked	Pass

1.1.2.9 Functional Testing 9: Export Research Results (Module 3)

Objective: To ensure users can export research results in various formats.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Export full research as PDF	Click "Export Research as PDF"	PDF file generated with all research topics, titles, summaries, and source URLs	PDF downloaded (1.2MB), contains all 12 topics	Pass
2	Export filtered research as PDF	Apply filter "Last 30 days" (5 topics), then export	PDF contains only filtered results (5 topics)	PDF contains 5 topics (filtered correctly)	Pass
3	Export as DOCX format	Click "Export as DOCX"	DOCX file downloaded with editable format	DOCX downloaded, can be opened in Microsoft Word	Pass
4	Export when no research exists	Delete all research, try export	Error: "No research data to export"	FAIL: System crashes with "Cannot read	Fail

				property 'map' of undefined" error	
5	Export as CSV format	Click "Export as CSV" (if supported)	CSV file with columns: Title, Summary, Source URL, Date, Relevance	CSV generated, 12 rows of data	Pass

1.1.2.10 Functional Testing 10: Session Management and Security

Objective: To ensure user sessions are managed securely and time out appropriately.

No.	Test Case	Attribute and value	Expected Result	Actual Result	Result
1	Session persists across page refresh	Login, refresh browser page	User remains logged in, dashboard reloads	User stays logged in, dashboard reloads correctly	Pass
2	Session expires after 24 hours of inactivity	Login, wait 25 hours, then refresh page	Session expired, redirect to login page	Redirected to login with message "Session expired"	Pass
3	Logout clears session	Click "Logout" button	Session token invalidated, redirect to login	Logged out, back button shows login page (not cached dashboard)	Pass
4	Cannot access protected route without login	Try to access /dashboard directly in URL without login	Redirect to login page	Redirected to login page	Pass
5	JWT stored securely	Inspect browser storage after login	JWT stored in HttpOnly cookie (not accessible via JavaScript)	FAIL: JWT stored in localStorage (vulnerable to XSS attacks)	Fail
6	Password not shown in browser console	Open developer tools, check console logs after login	No password printed in console	No password visible in console logs	Pass

1.1.3 Business Rules Testing

Decision table based testing technique is used to test business rules. The business rules were defined in FRs and Use Cases.

Decision based testing uses a systematic approach where inputs and outputs are provided in tabular form. It is a precise and compact way to model complicated logic. The table contains conditions and actions used for test cases where conditions as inputs and actions as outputs.

1.1.3.1 Business Rule Test 1: Account Lockout Policy (BR-A4)

Business Rule (BR-A4): After 5 consecutive failed login attempts, the account is temporarily locked for 15 minutes.

1.1.3.1.1 Decision Table:

Condition	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**	**Rule 5**	**Rule 6**
Failed attempt count < 5?	Y	Y	N	N	N	N
Failed attempt count = 5?	N	N	Y	Y	N	N
Account currently locked?	N	N	N	Y	Y	N
15 minutes passed?	-	-	-	N	Y	-
Actions						
Login allowed?	Y	Y	N	N	Y	Y
Lockout message shown?	N	N	Y	Y	N	N
Attempt counter increments?	Y	Y	N	N	N	N (resets)

1.1.3.1.2 Test Execution Table

Test Case ID	**Test Scenario**	**Test Data**	**Expected Result**	**Actual Result**	**Result**
BR_001	Attempt 1 (wrong password)	Email: "john@example.com", Password: "wrong1"	Login denied, counter = 1	Login denied, counter = 1	Pass
BR_002	Attempt 2 (wrong password)	Same email, another wrong password	Login denied, counter = 2	Login denied, counter = 2	Pass
BR_003	Attempt 5 (wrong password)	5th consecutive wrong attempt	Login denied, account locked for 15 minutes, lockout message shown	Account locked, message: "Too many failed attempts. Try again in 15 minutes"	Pass
BR_004	Attempt 6 (during lockout)	Attempt within 15 minutes of lockout	Login denied, lockout message shown	Login denied, remaining lockout time displayed	Pass
BR_005	Attempt after lockout period	Attempt after 15+ minutes	Login allowed, counter resets to 0	Login successful, counter reset	Pass

1.1.3.2 Business Rule Test 2: Free-Tier Monthly Episode Limit (BR-1)

Business Rule (BR-1): Free-tier users are limited to 3 episode plans per calendar month. Premium users have no limit.

1.1.3.2.1 Decision Table:

Condition	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**	**Rule 5**	**Rule 6**
User tier = Free?	Y	Y	Y	Y	N	N
Episodes used this month < 3?	Y	N	N	Y	-	-
New month started?	-	N	Y	-	-	-
User tier = Premium?	N	N	N	N	Y	Y
Actions						
Episode creation allowed?	Y	N	Y	Y	Y	Y
Error message shown?	N	Y	N	N	N	N
Upgrade prompt displayed?	N	Y	N	N	N	N

1.1.3.2.2 Test Execution Table:

Test Case ID	**Test Scenario**	**Test Data**	**Expected Result**	**Actual Result**	**Result**
BR_006	Free user, 0 episodes used	Click "Start New Episode"	Episode creation allowed	New episode created	Pass
BR_007	Free user, 2 episodes used	Click "Start New Episode"	Episode creation allowed (3rd allowed)	Episode created	Pass
BR_008	Free user, 3 episodes used	Click "Start New Episode"	Error: Monthly limit reached, upgrade prompt	Error displayed with upgrade link	Pass
BR_009	Free user, new month	After calendar reset, click "Start New Episode"	Episode creation allowed, counter resets	Episode created, counter = 1	Pass
BR_010	Premium user, 3 episodes used	Click "Start New Episode"	Episode creation allowed (no limit)	Episode created	Pass
BR_011	Premium user, 100 episodes used	Click "Start New Episode"	Episode creation allowed	FAIL: Episode creation blocked due to integer overflow bug	Fail

1.1.3.3 Business Rule Test 3: Maximum Bookmarks Per Episode (BR-7)

Business Rule (BR-7): A maximum of 10 topics may be bookmarked per episode.

1.1.3.3.1 Decision Table:

Condition	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**	**Rule 5**
Current bookmark count < 10?	Y	Y	N	N	N
Bookmark another topic?	Y	N	Y	N	Y
Unbookmark a topic first?	-	-	-	Y	N
Actions					
Bookmark allowed?	Y	-	N	Y	N
Error message shown?	N	-	Y	N	Y
Bookmark count changes	+1	No change	No change	-1 then +1	No change

1.1.3.3.2 Test Execution Table:

Test Case ID	Test Scenario	Test Data	Expected Result	Actual Result	Result
BR_012	Bookmark when count = 5	Click bookmark on 6th topic	Bookmark added, count = 6	Bookmark added, count = 6	Pass
BR_013	Bookmark when count = 9	Click bookmark on 10th topic	Bookmark added, count = 10	Bookmark added, count = 10	Pass
BR_014	Bookmark when count = 10	Click bookmark on 11th topic	Error: "Maximum 10 bookmarks. Remove one first."	Error displayed, bookmark not added	Pass
BR_015	Unbookmark then bookmark	Remove 1 bookmark, then bookmark new topic	Bookmark added, count remains 10	New bookmark added, count = 10	Pass

1.1.3.4 Business Rule Test 4: Minimum Bookmark Requirement for Question Generation (BR-8)

Business Rule (BR-8): At least one topic must be bookmarked before the Question Generator can be triggered.

1.1.3.4.1 Decision Table:

Condition	Rule 1	Rule 2	Rule 3
Bookmarked topics count = 0?	Y	N	N
Bookmarked topics count >= 1?	N	Y	Y
Click "Generate Questions"?	Y	Y	Y
Actions			
Question generation allowed?	N	Y	Y
Error message shown?	Y	N	N

1.1.3.4.2 Test Execution Table:

Test Case ID	Test Scenario	Test Data	Expected Result	Actual Result	Result
BR_016	No bookmarked topics	Bookmark count = 0, click "Generate Questions"	Error: "Please bookmark at least 1 topic first"	Error shown, generation blocked	Pass
BR_017	1 bookmarked topic	Bookmark count = 1, click "Generate Questions"	Question generation starts	Questions generated	Pass
BR_018	5 bookmarked topics	Bookmark count = 5, click "Generate Questions"	Question generation starts	Questions generated	Pass

1.1.3.5 Business Rule Test 5: Source Categories Based on User Tier (BR-6)

Business Rule (BR-6): Premium users receive research from 5 source categories; free-tier users receive from 2 source categories.

1.1.3.5.1 Decision Table:

Condition	**Rule 1**	**Rule 2**	**Rule 3**
User tier = Free?	Y	N	N
User tier = Premium?	N	Y	Y
Topic research triggered?	Y	Y	Y
Actions			
News category included?	Y	Y	Y
Academic category included?	N	Y	Y
Social Media category included?	N	Y	Y
Blogs category included?	Y	Y	Y
Podcasts category included?	N	Y	Y
Total source categories	2	5	5

1.1.3.5.2 Test Execution Table:

Decision Table:	**Decision Table:**	**Decision Table:**	**Decision Table:**	**Decision Table:**	**Decision Table:**
BR_019	Free user research	Niche: "Technology", Free account	Results from News and Blogs only	Research shows News and Blog sources only	Pass
BR_020	Premium user research	Niche: "Technology", Premium account	Results from all 5 source categories	Research shows 5 source categories	Pass

1.1.3.6 Business Rule Test 6: Guest Limit Per Episode (BR-3)

Business Rule (BR-3): A maximum of 3 guests may be added per episode.

1.1.3.6.1 Decision Table:

Condition	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5
Current guest count = 0?	Y	N	N	N	N
Current guest count = 1?	N	Y	N	N	N
Current guest count = 2?	N	N	Y	N	N
Current guest count = 3?	N	N	N	Y	N
Add another guest?	Y	Y	Y	Y	N
Actions					
Guest addition allowed?	Y	Y	Y	N	-
Error message shown?	N	N	N	Y	N

1.1.3.6.2 Test Execution Table:

Test Case ID	Test Scenario	Test Data	Expected Result	Actual Result	Result

BR_021	Add 1st guest	Episode has 0 guests, add guest	Guest added successfully	Guest added, count = 1	Pass
BR_022	Add 2nd guest	Episode has 1 guest, add guest	Guest added successfully	Guest added, count = 2	Pass
BR_023	Add 3rd guest	Episode has 2 guests, add guest	Guest added successfully	Guest added, count = 3	Pass
BR_024	Add 4th guest	Episode has 3 guests, add 4th guest	Error: "Maximum 3 guests per episode"	FAIL: Allows adding 4th guest (UI validation missing)	Fail

1.1.3.7 Business Rule Test 7: Password Strength Requirements (BR-A1)

Business Rule (BR-A1): Passwords must be at least 8 characters and include one uppercase letter, one number, and one special character.

1.1.3.7.1 Decision Table:

Condition	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**	**Rule 5**	**Rule 6**	**Rule 7**	**Rule 8**
Length >= 8?	Y	N	Y	Y	Y	Y	Y	Y
Contains uppercase?	Y	Y	N	Y	Y	Y	Y	Y
Contains lowercase?	Y	Y	Y	N	Y	Y	Y	Y
Contains number?	Y	Y	Y	Y	N	Y	Y	Y
Contains special char?	Y	Y	Y	Y	Y	N	Y	Y
Actions								
Password valid?	Y	N	N	N	N	N	N	N
Error message shown?	N	Y	Y	Y	Y	Y	Y	Y

1.1.3.7.2 Test Execution Table:

Test Case ID	**Test Scenario**	**Test Data**	**Expected Result**	**Actual Result**	**Result**
BR_025	Strong password	"PodCraft@2025"	Valid (True)	True	Pass
BR_026	Too short	"Pod@1"	Invalid - "Minimum 8 characters"	"Minimum 8 characters"	Pass
BR_027	No uppercase	"podcraft@2025"	Invalid - "Requires uppercase letter"	"Requires uppercase letter"	Pass
BR_028	No lowercase	"PODCRAFT@2025"	Invalid - "Requires	FAIL: Shows "Requires	Fail

			lowercase letter"	uppercase" instead	
BR_029	No number	"PodCraft@"	Invalid - "Requires number"	"Requires number"	Pass
BR_030	No special char	"PodCraft2025"	Invalid - "Requires special character"	"Requires special character"	Pass
BR_031	Empty password	""	Invalid - "Password cannot be empty"	"Password cannot be empty"	Pass

1.1.3.8 Business Rule Test 8: Research Cache Duration (BR-5)

Business Rule (BR-5): Research results are cached for 24 hours per niche-topic combination to reduce API costs.

1.1.3.8.1 Decision Table:

Condition	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**
Cache exists for (niche, topic)?	N	Y	Y	Y
Cache age < 24 hours?	-	Y	N	-
Cache age >= 24 hours?	-	N	Y	-
Cache manually cleared?	-	-	-	Y
Actions				
Use cached results?	N	Y	N	N
Call external API?	Y	N	Y	Y
Update cache with new results?	Y	N	Y	Y

1.1.3.8.2 Test Execution Table:

Test Case ID	**Test Scenario**	**Test Data**	**Expected Result**	**Actual Result**	**Result**
BR_032	First search (no cache)	Niche="Tech", Topic="AI"	API called, results cached	API called (2800ms), cache created	Pass
BR_033	Same search within 24h	Same niche+topic within 24h	Cache hit, results from cache (<100ms)	Cache hit (45ms)	Pass
BR_034	Same search after 24h	Same niche+topic after 25h	Cache miss, API called, cache updated	API called, new cache created	Pass
BR_035	Different topic, same niche	Niche="Tech", Topic="Blockchain"	New cache entry created	FAIL: Returns cached results from "AI" (cache key collision)	Fail

1.1.3.9 Business Rule Test 9: Session Expiration (BR-A3)

Business Rule (BR-A3): Session tokens expire after 24 hours of inactivity.

1.1.3.9.1 Decision Table:

Condition	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**
User logged in?	Y	Y	Y	N
Session age < 24 hours?	Y	N	-	-
Session age >= 24 hours?	N	Y	-	-
User active within 24 hours?	-	-	Y	-
User inactive for 24+ hours?	-	-	N	-
Actions				
Session valid?	Y	N	Y	N
Redirect to login?	N	Y	N	N
Session extended?	N	N	Y	N

1.1.3.9.2 Test Execution Table:

Test Case ID	**Test Scenario**	**Test Data**	**Expected Result**	**Actual Result**	**Result**
BR_036	Active session within 24h	Login, refresh after 12 hours	Session valid, dashboard loads	Dashboard loads correctly	Pass
BR_037	Session expired (25 hours)	Login, wait 25 hours, refresh	Session expired, redirect to login	Redirected to login with message	Pass
BR_038	Extended session with activity	Login, perform action at 23h 55m	Session extended for another 24h	Session extended, no logout	Pass

1.1.3.10 Business Rule Test 10: Episode Duration Limits

Business Rule (BR-12): Minimum time per question is 1 minute; maximum is 15 minutes

1.1.3.10.1 Decision Table:

Condition	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**	**Rule 5**
Duration per question (minutes)	0.5	1	15	16	10
Actions					
Duration valid?	N	Y	Y	N	Y
Error message shown?	Y	N	N	Y	N

1.1.3.10.2 Test Execution Table:

Test Case ID	**Test Scenario**	**Test Data**	**Expected Result**	**Actual Result**	**Result**
BR_039	Duration less than 1 min	0.5 minutes (30 seconds)	Error: "Minimum 1 minute per question"	Error displayed	Pass

BR_040	Duration exactly 1 min	1 minute	Valid, timestamp assigned	Timestamp = 1:00	Pass
BR_041	Duration exactly 15 min	15 minutes	Valid, timestamp assigned	Timestamp = 15:00	Pass
BR_042	Duration greater than 15 min	16 minutes	Error: "Maximum 15 minutes per question"	FAIL: Allows 16 minutes without error	Fail
BR_043	Duration within limits	10 minutes	Valid, timestamp assigned	Timestamp = 10:00	Pass

1.1.4 Integration Testing

Integration testing verifies that different modules of the system work together correctly. Unlike unit testing (which checks isolated functions) and functional testing (which checks features from a user's perspective), integration testing focuses on the interfaces, linkages, and data flow between modules developed by different team members.

Since FYPs are team-based, integration testing is essential to ensure that the combined work of individual members forms a functioning system.

For PodCraft, the following integration scenarios demonstrate how modules interact and exchange data:

- Integration 1: Module 1 (Authentication) ↔ Module 2 (User Intent & Guest Analyzer)
- Integration 2: Module 2 (User Intent & Guest Analyzer) ↔ Module 3 (Topic Research Agent)
- Integration 3: Module 1 ↔ Module 2 ↔ Module 3 (End-to-End Workflow)
- Integration 4: Frontend (React) ↔ Backend (Node.js/Express) API
- Integration 5: Backend ↔ OpenAI API for Research Generation
- Integration 6: Database Integration (User → Episode → Research)

1.1.4.1 Integration Testing 1: User Authentication ↔ Episode Creation Flow

Testing Objective: To ensure the interface between Module 1 (User Authentication & Profile Management) and Module 2 (User Intent & Guest Analyzer) is running correctly, specifically that an authenticated user can create an episode and that user identity is preserved throughout.

No.	**Test case/Test script**	**Attribute and value**	**Expected result**	**Actual result**	**Result**
1	Login → Create new episode (Module 1 ↔ Module 2)	User: john@example.com logs in, then clicks "Start New Episode"	User remains logged in, Episode creation screen loads with user's profile data	Dashboard loads with user name "John Podcaster" and tier "Free" displayed	Pass

			(name, email, tier)		
2	Login → Submit episode intent → Verify user_id linkage	After login, user selects niche "Technology", enters topic "AI Ethics", clicks "Analyze Intent"	Episode intent record saved in database with correct user_id foreign key linking to authenticated user	Database query shows episode.user_id = user_12345 (matches logged-in user)	Pass
3	Login → Save episode → Logout → Login → Retrieve saved episode (Session persistence across modules)	Complete episode plan, logout, login again with same credentials	Saved episode appears in user's episode history with all data intact	Episode list shows previously created episode with correct research data	Pass
4	User A cannot access User B's episodes (Data isolation across modules)	User A (john@example.com) logs in, tries to access URL of User B's episode (/episodes/ep_002)	Access denied, 403 Forbidden error or redirect to dashboard	Error: "You do not have permission to view this episode"	Pass
5	JWT expiration handling across sessions (Auth ↔ Session Management)	User logs in, waits 25 hours (token expired), then tries to access dashboard	Session expired, redirect to login page with message	FAIL: Redirects to login but no "session expired" message shown	Fail

1.1.4.2 Integration Testing 2: User Intent & Guest Analyzer ↔ Topic Research Agent

Testing Objective**:** To ensure the interface between Module 2 (User Intent & Guest Analyzer) and Module 3 (Topic Research Agent) is running correctly, specifically that the research agent receives the correct parameters and returns appropriate results.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Niche input → Topic Research trigger (Module 2 → Module 3)	User selects niche: "Business & Entrepreneurship", topic: "Remote Work", clicks "Analyze Intent"	Topic Research Agent receives "Business & Entrepreneurship" and "Remote Work" as parameters, returns business-related topics within 30 seconds	Research results: "Remote Work Trends 2025", "Hybrid Work Models", "Employee Productivity" (12 topics, 28 seconds)	Pass
2	Guest name input → Guest-specific research (Module 2 → Module 3)	User enters guest: "Elon Musk" instead of topic, niche: "Technology"	Topic Research Agent receives "Elon Musk" as search parameter, returns topics about Tesla, SpaceX, X.AI, Neura link	Research results: "Tesla Cybertruck Updates", "SpaceX Mars Mission 2026", "Neuralink Progress Report"	Pass
3	Combined niche + guest input (Multi-parameter integration)	Niche: "Artificial Intelligence", Guest: "Sam Altman"	Topic Research Agent combines both parameters, returns AI topics with OpenAI/Altman relevance	Results: "GPT-5 Release Date", "OpenAI Governance Structure", "Future of AGI"	Pass
4	Invalid/empty input propagation (Error handling across modules)	User submits with empty niche and empty topic (violates FR2.2)	Module 2 (Intent Analyzer) returns validation error BEFORE reaching Module 3 (Topic Research Agent)	Research agent not called, validation error: "Please select a niche and enter a topic or guest name"	Pass
5	Research results stored with correct episode linkage (Data flow integrity)	Research completes successfully for episode "ep_003"	Research results saved in database with episode_id correctly linking to the episode intent record	Query shows research_results.episode_id = ep_003, relationship verified	Pass
6	Research timeout handling (Module 3	OpenAI API response takes > 30 seconds	Module 3 returns timeout error, Module 2 displays friendly error	FAIL: System hangs indefinitely, no timeout implemented	Fail

	↔ External API)	(simulated slow network)	message to user, allows retry		

1.1.4.3 Integration Testing 3: End-to-End Module 1 → Module 2 → Module 3 Workflow

Testing Objective: To test the complete end-to-end integration across all three modules, ensuring data flows correctly from authentication through intent analysis to topic research.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Complete workflow: Register → Login → Create Profile → Submit Intent → Research (Full pipeline)	New user: Name="E2E Tester", Email="e2e@example.com", Password="Test@1234", then Niche="Technology", Topic="AI in Healthcare"	All steps complete successfully, episode saved with correct user association, intent stored, research results displayed	Registration success → Login success → Profile created → Intent analyzed → Research completed (45 seconds total)	Pass
2	Saved episode retrieval after research (Persistence across modules)	After completing research, user saves episode, logs out and back in	Episode appears in history, research results load correctly when episode is opened	Episode loaded with previous research results intact (12 topics)	Pass
3	Free tier limit integration across modules (Business rule enforcement)	Free user creates 3 episodes (with research), attempts 4th episode	Module 1 (auth) identifies tier, Module 2 blocks episode creation BEFORE triggering Module 3 (research), saving API costs	Error shown: "Monthly limit reached (3/3). Upgrade to continue." Research agent not called	Pass
4	Bookmarked topics passed from Module 3 to Question	User bookmarks 3 topics in research results, then clicks "Generate Questions"	Bookmarked topics prioritized and passed to	Bookmarked topics flagged with priority=true	Pass

	Generator (Module 3 → Module 4 readiness)		Question Generator module	in data passed to next module	
5	Concurrent user sessions (Multi-user integration)	User A and User B simultaneously create episodes with different niches	Both sessions isolated, each user's research results correct for their niche	User A gets Technology topics, User B gets Business topics, no cross-contamination	Pass
6	Network interruption during research (Resilience testing)	User submits intent, research starts, then network disconnects	System displays error message, saves partial progress, allows retry	FAIL: Partial research results lost, no auto-save implemented	Fail

1.1.4.4 Integration Testing 4: Frontend (React) ↔ Backend (Node.js/Express) API

Testing Objective: To ensure the API layer correctly handles requests from the React frontend, validates authentication, and returns appropriate responses.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Frontend calls POST /api/auth/login (Auth API)	Body: { email: "john@example.com", password: "Pass@1234" }	Backend validates credentials, returns 200 OK with JWT token	Response: { success: true, token: "eyJhbGciOiJIUzI1..." }	Pass
2	Frontend calls POST /api/episodes with valid JWT (Episode creation API)	Header: Authorization: Bearer {token}, Body: { niche: "Technology", topic: "AI Ethics" }	Backend validates JWT, creates episode, returns episode_id	Response: { success: true, episodeId: "ep_12345" }	Pass
3	Frontend calls POST /api/episodes/:id/research (Research API)	episodeId: "ep_12345", Header: Authorization: Bearer {token}	Backend calls OpenAI API, returns research results array	Response: { topics: [{ title: "...", summary: "..." }] } (12 topics)	Pass
4	Frontend calls API with invalid JWT (Security integration)	Header: Authorization: Bearer	Backend returns 401 Unauthorized, frontend	401 status received, frontend shows "Session expired. Please log in."	Pass

		"invalid_token_123 "	redirects to login		
5	Frontend calls API with missing required fields (Validation integration)	Body: { niche: "" } (empty niche)	Backend returns 400 Bad Request with validation errors	400 status, error: { field: "niche", message: "Niche is required" }	Pass
6	Frontend calls API with malformed JSON (Error handling)	Body: "{ niche: 'Technology', topic: 'AI' " (missing closing brace)	Backend returns 400 Bad Request with parse error	FAIL: Returns 500 Internal Server Error instead of 400	Fail
7	API rate limiting for free tier users (Integration with subscription module)	Free user makes 100 research requests in 1 minute	After limit exceeded, returns 429 Too Many Requests with retry-after header	429 status with Retry-After: 60 seconds	Pass

1.1.4.5 Integration Testing 5: Backend ↔ OpenAI API for Research Generation

Testing Objective: To ensure the backend correctly communicates with the OpenAI API, handles responses, and manages errors.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Backend calls OpenAI API with valid prompt (Happy path)	Prompt: "Generate 10 research topics about AI in Healthcare"	OpenAI returns 10 topics, backend parses and stores results	10 topics received, parsed, stored in database	Pass
2	Backend handles OpenAI API timeout (Error handling)	OpenAI API takes > 30 seconds to respond (simulated)	Backend retries up to 3 times with exponential backoff, then returns error to frontend	FAIL: No retry mechanism, immediately returns timeout error	Fail
3	Backend handles OpenAI API rate limit (429 error)	OpenAI returns 429 Too Many Requests	Backend implements exponential backoff (1s, 2s, 4s)	Retry after 2 seconds, success on 2nd attempt	Pass

			and retries successfully		
4	Backend caches OpenAI responses (BR-5 integration)	Same niche+topic requested twice within 24 hours	Second request uses cache instead of calling OpenAI API	Cache hit (45ms vs 2800ms), OpenAI API not called	Pass
5	Backend handles malformed OpenAI response (Data validation)	OpenAI returns malformed JSON (missing required fields)	Backend validates response, returns error to frontend with specific message	Error: "Invalid response from AI service. Please try again."	Pass
6	Backend truncates long inputs before sending to OpenAI (Cost optimization)	User submits 5000 character topic description	Backend truncates to 1000 characters before sending to OpenAI API	Truncated to 1000 chars, API call successful	Pass

1.1.4.6 Integration Testing 6: Database Integration – User ↔ Episode ↔ Research Tables

Testing Objective: To ensure the database schema correctly links users, episodes, and research results with proper foreign key constraints and cascading behavior.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Create user → Create episode → Add research (Sequential data insertion)	Insert user record, then episode with user_id FK, then research with episode_id FK	All three records linked correctly in database, JOIN query returns all data	Query: SELECT * FROM users u JOIN episodes e ON u.id=e.user_id JOIN research r ON e.id=r.episode_id WHERE u.id='user_123' → returns 1 row	Pass
2	Delete user with episodes (Cascade delete)	Delete user account (user_id = 'user_123')	Episodes and research results cascade delete (or are orphaned based on design)	All associated episodes and research removed from database (CASCADE set)	Pass
3	Delete episode with research (Cascade delete)	Delete only the episode (episode_id = 'ep_123')	Research results cascade delete or are marked orphaned	Research records no longer accessible via episode query (deleted)	Pass
4	Attempt to create episode with non-	Insert episode with user_id =	Foreign key constraint	Database error: "foreign key violation"	Pass

	existent user_id (Referential integrity)	'user_99999' (no such user)	violation, insert rejected		
5	Attempt to create research with non-existent episode_id (Referential integrity)	Insert research with episode_id = 'ep_99999' (no such episode)	Foreign key constraint violation, insert rejected	Database error: "foreign key violation"	Pass
6	Concurrent database transactions (Multi-user data integrity)	User A and User B simultaneously create episodes with same title	Both transactions succeed, episodes stored with unique IDs	Both episodes created, no data corruption	Pass
7	Database backup and recovery (Reliability integration)	Simulate database crash, restore from backup	All user data recovered within RPO (6 hours) and RTO (1 hour)	FAIL: Backup not tested, no automated recovery procedure documented	Fail

1.1.4.7 Integration Testing 7: Firebase Authentication ↔ JWT Session Management

Testing Objective: To ensure Firebase Auth (Google OAuth) correctly integrates with the local JWT session management system.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Google OAuth login → JWT creation (Third-party auth integration)	User clicks "Continue with Google", selects Google account	Firebase returns user data, backend creates local JWT, returns to frontend	JWT created, user logged in, dashboard loaded	Pass
2	Google OAuth user without existing account (Auto-registration)	New user logs in with Google for first time	System automatically creates local user account (no password required)	Account created, user profile populated from Google data	Pass
3	Google OAuth user with existing email (Account linking)	User registered with email "john@example.com" previously, now logs in with Google using same email	System links Google account to existing local account	Accounts linked, user can log in with either method	Pass

4	Firebase token expiration handling	Firebase token expires (after 1 hour), user attempts action	Frontend refreshes token automatically, user continues seamlessly	FAIL: Token refresh fails, user forced to logout	Fail
5	Logout clears both Firebase and local sessions	User clicks "Logout"	Both Firebase session and local JWT invalid	Both sessions cleared, user redirected to login	Pass

1.2 Mobile App:

1.2.1 Unit Testing

Unit testing verifies the smallest testable components of the software (e.g., individual functions, methods, or classes) in isolation. The purpose is to ensure that each unit performs as expected, independent of the full system. For PodCraft, unit tests were written for core backend utility functions using the Detailed Test Case Format as per lecture guidelines.

1.2.1.1 MODULE 1: User Authentication & Profile Management:

1.2.1.1.1 Unit Test Case 1.1: validateEmail() Function

Test case ID:	UT_1_1_001
Test case description:	To verify that email validation correctly identifies valid and invalid email formats.
Preconditions:	Email validation function is imported.
Test steps:	1. Call with valid email 2. Call with missing @ 3. Call with missing domain 4. Call with spaces 5. Call with empty string 6. Call with special chars only
Test data:	1. "user@example.com" 2. "userexample.com" 3. "user@" 4. "user @example.com" 5. "" 6. "@#$%^&*()"
Expected Result:	1. True 2. False 3. False 4. False

	5. False 6. False
Actual Result:	1. True 2. False 3. False 4. False 5. False 6. False
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

1.2.1.1.2 Unit Test Case 1.2: matchPassword() Function

Test case ID:	UT_1_2_001
Test case description:	To verify that password and confirm-password matching enforces exact, case-sensitive equality before submission.
Preconditions:	Sign-up form imports matchPassword utility.
Test steps:	1. Identical strong password 2. Different case 3. Different length 4. Confirm field empty 5. Both fields empty 6. Trailing space mismatch
Test data:	1. ("Pass@1234","Pass@1234") 2. ("Pass@1234","pass@1234") 3. ("Pass@1234","Pass@123") 4. ("Pass@1234","") 5. ("","") 6. ("Pass@1234 ","Pass@1234")
Expected Result:	1. True 2. False 3. False 4. False 5. False 6. False
Actual Result:	1. True 2. False 3. False 4. False 5. False 6. False
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)

Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

1.2.1.1.3 Unit Test Case 1.3: generateJWT() Function

Test case ID:	UT_1_3_001
Test case description:	To verify that the JWT generator produces a well-formed three-part token (header.payload.signature) and that it decodes back to the same user identifier.
Preconditions:	JWT_SECRET set in /app/backend/.env; pyjwt library imported.
Test steps:	1. UUID user id 2. User id with special chars 3. Very long (200-char) user id 4. Simple short id
Test data:	1. "11111111-2222-3333-4444-555555555555" 2. "user_with_special_!@#$%" 3. "a"*200 4. "simple"
Expected Result:	1. Three-part token, decodes correctly 2. Three-part token, decodes correctly 3. Three-part token, decodes correctly 4. Three-part token, decodes correctly
Actual Result:	1. Three-part token, decodes correctly 2. Three-part token, decodes correctly 3. Three-part token, decodes correctly 4. Three-part token, decodes correctly
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

1.2.1.1.4 Unit Test Case 1.4: hashPassword() and verifyPassword() Functions

Test case ID:	UT_1_4_001
Test case description:	To verify that bcrypt hashing produces a $2b$ prefixed hash and that verification returns True for the original password and False for any tampered password.
Preconditions:	bcrypt library imported; helpers _hash_password and _verify_password available.
Test steps:	1. Short password 2. Strong password 3. Unicode emoji password 4. 60-character password 5. 100-character password (greater than bcrypt 72-byte limit)

Test data:	1. "123" 2. "Pass@1234" 3. "□□podcast" 4. "a"*60 5. "a"*100
Expected Result:	1. Hash starts with $2b$ and verify True/False 2. Hash starts with $2b$ and verify True/False 3. Hash starts with $2b$ and verify True/False 4. Hash starts with $2b$ and verify True/False 5. Hash starts with $2b$ and verify True/False
Actual Result:	1. Pass — hash and verify correct 2. Pass — hash and verify correct 3. Pass — hash and verify correct 4. Pass — hash and verify correct 5. FAIL — bcrypt silently truncates input to 72 bytes; "a"*100 and "a"*100+"x" both verify True. Mitigation: sign-up form enforces 64-char max password length.
Test Case Status:	Fail
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

1.2.1.1.5 Unit Test Case 1.5: POST /api/auth/login Endpoint

Test case ID:	UT_1_5_001
Test case description:	To verify that the login endpoint returns 200 with a token+user object for valid credentials, and 401 Unauthorized for invalid credentials.
Preconditions:	Backend running; seeded account awais@gmail.com / 123 exists.
Test steps:	1. Login with seeded account 2. Login with wrong password 3. Login with unknown email
Test data:	1. {"email":"awais@gmail.com","password":"123"} 2. {"email":"awais@gmail.com","password":"wrong"} 3. {"email":"ghost@nowhere.io","password":"123"}
Expected Result:	1. 200 with token+user object 2. 401 Unauthorized 3. 401 Unauthorized
Actual Result:	1. 200 with token+user object 2. 401 Unauthorized 3. 401 Unauthorized
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

1.2.1.1.6 Unit Test Case 1.6: POST /api/auth/signup Endpoint

Test case ID:	UT_1_6_001
Test case description:	To verify that the signup endpoint creates a new account, rejects duplicate emails, and rejects malformed emails.
Preconditions:	Backend running.
Test steps:	1. Signup with unique email 2. Signup with already-registered email 3. Signup with malformed email
Test data:	1. {"email":"u_<rand>@test.com","password":"Pass@1234","name":"QA Bot"} 2. {"email":"awais@gmail.com","password":"123","name":"Awais"} 3. {"email":"not-an-email","password":"Pass@1234","name":"X"}
Expected Result:	1. 200 with token and user object 2. 400 Bad Request — detail: Email already registered 3. 422 Unprocessable Entity
Actual Result:	1. 200 with token and user object 2. 400 Bad Request — detail: Email already registered 3. 422 Unprocessable Entity
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

Unit Test Case Summary — Module 1: User Authentication & Profile Management

Test Case ID	Function / Endpoint	Description	Expected Result	Actual Result	Status
UT_1_1_001	validateEmail()	Email format validation	True/False per test data	True/False per test data	Pass
UT_1_2_001	matchPassword()	Password equality check	True/False per test data	True/False per test data	Pass
UT_1_3_001	generateJWT()	JWT generation and decode	3-part token, decodes correctly	3-part token, decodes correctly	Pass
UT_1_4_001	hashPassword() / verifyPassword()	bcrypt hash and verify	Hash $2b$, verify True/False	Pass for cases 1-4; FAIL case 5 (72-byte truncation)	Fail
UT_1_5_001	POST /api/auth/login	Login endpoint validation	200+token or 401	200+token or 401	Pass
UT_1_6_001	POST /api/auth/signup	Signup endpoint validation	200, 400, or 422 per case	200, 400, or 422 per case	Pass

1.2.1.2 MODULE 2: Episode Setup & Guest Management:

1.2.1.2.1 Unit Test Case 2.1: validateNiche() Function

Test case ID:	UT_2_1_001
Test case description:	To verify that the niche/topic input passes validation when non-empty and rejects empty input at the wizard layer.
Preconditions:	Wizard step-1 imports validateNiche helper.
Test steps:	1. Niche provided 2. Empty niche 3. Whitespace-only niche
Test data:	1. "Indie SaaS founders" 2. "" 3. " "
Expected Result:	1. True 2. False 3. False
Actual Result:	1. True 2. False 3. False
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

1.2.1.2.2 Unit Test Case 2.2: buildEpisodePrompt() Function

Test case ID:	UT_2_2_001
Test case description:	To verify that the AI prompt builder embeds every wizard input (niche, guest, goal, tone, length, format) into the final prompt sent to GPT-5.2.
Preconditions:	Episode object with all wizard fields populated.
Test steps:	1. Build prompt with full input
Test data:	1. {niche:"Indie SaaS founders", guest_name:"Pieter Levels", goal:"Educational", tone:"Casual", length:"medium", format:"interview"}
Expected Result:	1. Prompt contains all six attributes in correct slots
Actual Result:	1. Prompt contains all six attributes in correct slots
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

Unit Test Case Summary — Module 2: Episode Setup & Guest Management

Test Case ID	Function / Endpoint	Description	Expected Result	Actual Result	Status
UT_2_1_001	validateNiche()	Niche/topic input validation	True/False per test data	True/False per test data	Pass
UT_2_2_001	buildEpisodePrompt()	AI prompt builder with all fields	All 6 attributes present in prompt	All 6 attributes present in prompt	Pass

1.2.1.3 MODULE 3: Topic Research, Question, Hook & Outline Generation:

1.2.1.3.1 Unit Test Case 3.1: extractJSON() Function

Test case ID:	UT_3_1_001
Test case description:	To verify that the LLM response parser strips Markdown code fences and parses valid JSON from a wrapped response.
Preconditions:	GPT-5.2 returns content sometimes wrapped in ```json fences.
Test steps:	1. JSON without fences 2. JSON wrapped in ```json fences 3. JSON wrapped in ``` fences
Test data:	1. {"title":"Ep"} 2. ```json {"title":"Ep"} ``` 3. ``` {"title":"Ep"} ```
Expected Result:	1. {"title":"Ep"} parsed 2. {"title":"Ep"} parsed 3. {"title":"Ep"} parsed
Actual Result:	1. {"title":"Ep"} parsed 2. {"title":"Ep"} parsed 3. {"title":"Ep"} parsed
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

1.2.1.3.2 Unit Test Case 3.2: distributeTimestamps() Function

Test case ID:	UT_3_2_001
Test case description:	To verify that the timestamp distributor produces N ascending mm:00 markers beginning at 00:00 across a target episode duration.
Preconditions:	Outline section count and total length in minutes are known.
Test steps:	1. 45 minutes across 6 segments
Test data:	1. total_minutes=45, segments=6
Expected Result:	1. Six markers, ascending, first is 00:00
Actual Result:	1. Six markers, ascending, first is 00:00
Test Case Status:	Pass

Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

1.2.1.3.3 Unit Test Case 3.3: rankHooks() Function

Test case ID:	UT_3_3_001
Test case description:	To verify that the hook ranker orders hook objects by descending text length (used as a proxy for development depth).
Preconditions:	Three hook objects of differing text length.
Test steps:	1. Sort hooks
Test data:	1. [{"text":"Short"},{"text":"A much longer and more developed hook line"},{"text":"Mid length"}]
Expected Result:	1. Top hook has the longest text; bottom hook has the shortest
Actual Result:	1. Top hook has the longest text; bottom hook has the shortest
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

1.2.1.3.4 Unit Test Case 3.4: exportOutlineTXT() Function

Test case ID:	UT_3_4_001
Test case description:	To verify that the TXT exporter renders the episode title and each outline section with its duration and purpose.
Preconditions:	Episode object with title and at least one outline section.
Test steps:	1. Export single-section outline
Test data:	1. {"title":"Test Ep","plan":{"outline":[{"section":"Cold Open","duration_min":2,"purpose":"Hook"}]}}
Expected Result:	1. Output contains "# Test Ep", "Cold Open" and "2m"
Actual Result:	1. Output contains "# Test Ep", "Cold Open" and "2m"
Test Case Status:	Pass
Created By:	Awais (PodCraft Team)
Date of creation:	2026-05-09
Executed By:	Awais (PodCraft Team)
Date of execution:	2026-05-15

Unit Test Case Summary — Module 3: Topic Research, Question, Hook & Outline Generation

Test Case ID	Function / Endpoint	Description	Expected Result	Actual Result	Status
UT_3_1_001	extractJSON()	Strip Markdown fences and parse JSON	Dict parsed correctly	Dict parsed correctly	Pass

UT_3_2_001	distributeTimestamps()	Produce N ascending mm:00 markers	6 ascending markers, first 00:00	6 ascending markers, first 00:00	Pass
UT_3_3_001	rankHooks()	Sort hooks by descending text length	Longest hook first	Longest hook first	Pass
UT_3_4_001	exportOutlineTXT()	TXT export with title and sections	Contains title, section name, duration	Contains title, section name, duration	Pass

1.2.2 Functional Testing

Functional testing validates that the system modules work correctly as a whole, ensuring that the developed system meets its specifications and requirements. Unlike unit testing, which focuses on internal functions, functional testing evaluates user-facing features through the UI or APIs.

1.2.2.1 Functional Testing 1: User Registration with Different Scenarios

Objective: To ensure that user registration works correctly for valid data and shows appropriate errors for invalid data.

No.	Test Case Attribute and value	Expected Result	Actual Result	Result
1	Register with valid credentials. Name: "John Podcaster", Email: "john@example.com", Password: "Pass@1234"	Account created, JWT returned, redirect to dashboard	Account created, JWT returned, redirect to dashboard	Pass
2	Register with existing email. Email: "awais@gmail.com"	Error: "Email already registered"	Error: "Email already registered"	Pass
3	Register with invalid email format. Email: "notanemail"	Inline error message and 422 from API	Inline error message and 422 from API	Pass
4	Register with missing name. Name: ""	Error: "Please fill in all fields"	Error: "Please fill in all fields"	Pass
5	Register with empty password. Password: ""	Error: "Please fill in all fields"	Error: "Please fill in all fields"	Pass

1.2.2.2 Functional Testing 2: User Login with Different Scenarios

Objective: To verify that login works with valid credentials and rejects invalid ones.

No.	Test Case Attribute and value	Expected Result	Actual Result	Result
1	Login with seeded credentials. Email: "awais@gmail.com", Password: "123"	Dashboard loads with greeting "Hey Awais"	Dashboard loads with greeting "Hey Awais"	Pass

2	Login with wrong password. Email: "awais@gmail.com", Password: "wrong"	Alert: "Invalid email or password"	Alert: "Invalid email or password"	Pass
3	Login with unknown email. Email: "ghost@nowhere.io"	Alert: "Invalid email or password"	Alert: "Invalid email or password"	Pass
4	Login with whitespace-padded email. Email: " awais@gmail.com "	Email trimmed and login succeeds	Email trimmed and login succeeds	Pass

1.2.2.3 Functional Testing 3: Episode Creation Wizard (Three Steps)

Objective: To verify that the multi-step wizard collects required inputs and triggers the AI generation pipeline.

No.	Test Case Attribute and value	Expected Result	Actual Result	Result
1	Step 1: enter niche "Indie SaaS founders", select goal "Educational", tap Continue	Step indicator advances to 2 of 3	Step indicator advances to 2 of 3	Pass
2	Step 1: leave niche empty, tap Continue	Alert: "Niche required. Tell us your podcast niche first."	Alert: "Niche required. Tell us your podcast niche first."	Pass
3	Step 2: pick tone Casual, length Medium, format Interview	Selected pills highlight; step advances to 3 of 3	Selected pills highlight; step advances to 3 of 3	Pass
4	Step 3: review summary, tap Generate Episode Plan	Waveform loader displayed, results screen opens with five populated tabs	Waveform loader displayed ~48s, results screen opens with five tabs (5 topics, 10 questions, 5 hooks, 7 outline sections, 7 timestamps)	Pass

1.2.2.4 Functional Testing 4: Results Screen — Tabs, Copy, Regenerate, Share

Objective: To verify that all interactive controls on the results screen behave as designed.

No.	Test Case Attribute and value	Expected Result	Actual Result	Result
1	Tap each pill tab (Topics, Questions, Hooks, Outline, Timestamps)	Content area swaps to the chosen section	Content area swaps correctly	Pass
2	On Hooks tab, tap Regenerate	Only the hooks array changes; topics and outline preserved	Only the hooks array changes; topics and outline preserved	Pass
3	Tap any card	Toast: "Copied. Item copied to clipboard"	Toast: "Copied. Item copied to clipboard"	Pass

4	Tap bottom button Copy all and paste elsewhere	Full Markdown plan with all sections is pasted	Full Markdown plan with all sections is pasted	Pass
5	Tap bottom button Export / Share	Native OS share sheet opens with the plan text	Native OS share sheet opens with the plan text	Pass
6	Pill tabs render labels on iOS Expo Go	Each pill shows icon, label and count badge	Initial bug: labels collapsed on iOS Expo Go due to flex gap; fixed by switching to explicit margins and flexShrink:0, then re-verified	Pass

1.2.2.5 Functional Testing 5: Drafts Library

Objective: To verify that generated episodes persist and can be browsed and deleted.

No.	Test Case Attribute and value	Expected Result	Actual Result	Result
1	Open Drafts tab	Saved episodes listed by creation date, descending	Saved episodes listed by creation date, descending	Pass
2	Long-press a draft card	Confirmation dialog: "Delete draft?"	Confirmation dialog appears; on confirm, card removed from list	Pass
3	Pull down to refresh	List refreshes from API	List refreshes from API	Pass

1.2.2.6 Functional Testing 6: Profile and Sign Out

Objective: To verify the profile screen and logout flow.

No.	Test Case Attribute and value	Expected Result	Actual Result	Result
1	Open Profile tab	Shows user name, email and avatar initial	Shows user name, email and avatar initial	Pass
2	Tap Sign out and confirm	Token cleared from AsyncStorage; redirect to Login	Token cleared from AsyncStorage; redirect to Login	Pass

1.2.3 Business Rules Testing

Decision table based testing technique is used to test business rules. The business rules were defined in FRs and Use Cases. Decision based testing uses a systematic approach where inputs and outputs are provided in tabular form. It is a precise and compact way to model complicated

logic. The table contains conditions and actions used for test cases where conditions as inputs and actions as outputs.

1.2.3.1 Business Rule Test 1: Signup Validity Policy (BR-1)

Business Rule (BR-1): A new account can only be created when the email is well-formed, the password is non-empty, the name is non-empty, and the email is not already registered.

Decision Table:

Condition	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5
Email well-formed?	Y	N	Y	Y	Y
Password non-empty?	Y	Y	N	Y	Y
Name non-empty?	Y	Y	Y	N	Y
Email already used?	N	N	N	N	Y
Actions					
Account created?	Y	N	N	N	N
Validation error 422?	N	Y	Y	Y	N
Duplicate error 400?	N	N	N	N	Y

Test Execution Table:

Test Case ID	Test Scenario	Test Data	Expected Result	Actual Result	Result
BR_001	Valid unique signup	name="QA Bot", email="u_<rand>@test.com", password="Pass@1234"	Account created, 200 with JWT	Account created, 200 with JWT	Pass
BR_002	Malformed email	email="notanemail"	422 Unprocessable Entity	422 Unprocessable Entity	Pass
BR_003	Empty password	password=""	422 Unprocessable Entity	422 Unprocessable Entity	Pass
BR_004	Empty name	name=""	422 Unprocessable Entity	422 Unprocessable Entity	Pass
BR_005	Duplicate email	email="awais@gmail.com"	400 with detail "Email already registered"	400 with detail "Email already registered"	Pass

1.2.3.2 Business Rule Test 2: Episode Length Drives Outline Duration (BR-2)

Business Rule (BR-2): The chosen episode length controls the target total duration of the generated outline: short ~20 min, medium ~45 min, long ~75 min.

Decision Table:

Condition	Rule 1	Rule 2	Rule 3

length = short?	Y	N	N
length = medium?	N	Y	N
length = long?	N	N	Y
Actions			
Target ~20 min	Y	N	N
Target ~45 min	N	Y	N
Target ~75 min	N	N	Y

Test Execution Table:

Test Case ID	Test Scenario	Test Data	Expected Result	Actual Result	Result
BR_006	Generate short episode	length="short"	Sum of section durations ~20 min	Sum of section durations = 21 min	Pass
BR_007	Generate medium episode	length="medium"	Sum of section durations ~45 min	Sum of section durations = 45 min	Pass
BR_008	Generate long episode	length="long"	Sum of section durations ~75 min	Sum of section durations = 74 min	Pass

1.2.3.3 Business Rule Test 3: User-Scoped Episode Access (BR-3)

Business Rule (BR-3): A user can only retrieve, regenerate or delete episodes that belong to them. All other access attempts return 404.

Decision Table:

Condition	Rule 1	Rule 2	Rule 3
Valid JWT?	Y	Y	N
JWT belongs to owning user?	Y	N	-
Actions			
Return episode 200?	Y	N	N
Return 404 Not Found?	N	Y	N
Return 401/403 Unauthorized?	N	N	Y

Test Execution Table:

Test Case ID	Test Scenario	Test Data	Expected Result	Actual Result	Result
BR_009	Owner reads own episode	Owner JWT + own episode id	200 with episode	200 with episode	Pass
BR_010	Different user reads another user's episode	Other JWT + episode id of another user	404 Not Found	404 Not Found	Pass
BR_011	Request without JWT	No Authorization header	401 / 403	403 Forbidden	Pass

1.2.3.4 Business Rule Test 4: Regenerate Section Scope (BR-4)

Business Rule (BR-4): A regenerate request updates only the requested section of an episode; the section name must belong to a fixed allow-list.

Decision Table:

Condition	Rule 1	Rule 2
section in {topics, questions, hooks, outline, timestamps}?	Y	N
Actions		
Regenerate that section only?	Y	N
Return 400 Invalid section?	N	Y

Test Execution Table:

Test Case ID	Test Scenario	Test Data	Expected Result	Actual Result	Result
BR_012	Regenerate hooks	{"section":"hooks"}	Only hooks array changes; other sections preserved	Only hooks array changes; other sections preserved	Pass
BR_013	Regenerate unknown section	{"section":"bogus"}	400 Bad Request	400 Bad Request	Pass

1.2.4 Integration Testing

Integration testing verifies that different modules of the system work together correctly. Unlike unit testing (which checks isolated functions) and functional testing (which checks features from a user's perspective), integration testing focuses on the interfaces, linkages and data flow between modules developed by different team members. For PodCraft, the following integration scenarios demonstrate how modules interact and exchange data:

1.2.4.1 Integration Testing 1: User Authentication <-> Episode Creation Flow

Objective: To ensure the interface between Module 1 (User Authentication & Profile Management) and Module 2 (Episode Setup & Guest Management) operates correctly, specifically that an authenticated user can create an episode and that user identity is preserved throughout.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Login -> Open wizard (Module 1 <-> Module 2)	User awais@gmail.com logs in, then taps "Create New Episode"	User remains logged in, wizard opens with user identity in JWT	Dashboard loads with greeting "Hey Awais"; wizard opens; subsequent /api/episodes/generate	Pass

			header on every request	request carries the Bearer token	
2	Login -> Submit episode intent -> Verify user_id linkage	After login, user enters niche "Indie SaaS founders", goal Educational, then taps Generate	Generated episode persisted in MongoDB with correct user_id linking to the authenticated user	Database document shows episode.user_id equal to the authenticated user id	Pass
3	Login -> Generate -> Logout -> Login -> Retrieve saved episode	Complete episode plan, sign out from Profile tab, then log in again with the same credentials	Saved episode appears in the Drafts list with all sections intact	Drafts tab shows previously created episode with all five sections intact	Pass
4	Data isolation: User-A cannot read User-B's episode	User-A logs in, then issues GET /api/episodes/<User-B episode id>	404 Not Found or 403 Forbidden	404 Not Found	Pass

1.2.4.2 Integration Testing 2: Episode Service <-> GPT-5.2 (emergentintegrations.LlmChat)

Objective: To ensure that the backend correctly invokes the GPT-5.2 model, parses the JSON response and populates an Episode document.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Generate full plan	niche="Indie SaaS founders", goal="Educational", tone="Casual", length="medium", format="interview"	Plan returned within 30-60 seconds; JSON parses; five sections all non-empty	Plan returned in 48 seconds; parsing succeeded; 5 topics, 10 questions, 5 hooks, 7 outline sections and 7 timestamps generated	Pass
2	Regenerate one section	POST /api/episodes/<id>/regenerate {"section":"hooks"}	Only the hooks array is replaced; other	Only the hooks array is replaced; other	Pass

			sections preserved	sections preserved	
3	Response wrapped in Markdown code fences	LLM returned ```json ... ``` wrapper around payload	extract_json() strips fences and parses successfully	extract_json() strips fences and parses successfully	Pass

1.2.4.3 Integration Testing 3: Episode Service <-> MongoDB (motor)

Objective: To ensure that persistence operations (create, read, list, delete) work end-to-end and that internal ObjectId fields never leak into API responses.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Create episode -> list episodes	Generate one episode, then GET /api/episodes	Newly-created episode appears at the top of the list (sorted by created_at descending)	Newly-created episode appears at the top of the list	Pass
2	Delete episode -> read deleted id	DELETE /api/episodes/<id>, then GET /api/episodes/<id>	{"ok": true} followed by 404 Not Found	{"ok": true} followed by 404 Not Found	Pass
3	No internal ObjectId leak	Inspect every response body that contains episodes or users	No _id field present in any response	Verified: _id excluded by projection on every query and aggregation	Pass

1.2.4.4 Integration Testing 4: Frontend (Expo) <-> Backend (FastAPI) over HTTPS

Objective: To ensure that the mobile front-end correctly communicates with the deployed back-end without any hard-coded URLs and that the session persists across reloads.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Frontend uses EXPO_PUBLIC_BACKEND_URL only	Static code scan for hard-coded URLs	No hard-coded URLs, all traffic goes through the environment variable	Confirmed by automated deployment health check	Pass
2	CORS allows preview origin	Login from the Expo Go	Pre-flight OPTIONS passes; POST	Pre-flight OPTIONS passes; POST	Pass

		web preview	/api/auth/login returns 200	/api/auth/login returns 200	
3	JWT persisted across reloads (AsyncStorage)	Login, close and re-open the Expo Go app	User remains logged-in and lands on the Dashboard	User remains logged-in and lands on the Dashboard	Pass

1.3 Summary of Results

Test Category	**Total**	**Passed**	**Failed**
Unit Testing (parametrised cases)	33	32	1
Functional Testing	22	22	0
Business Rules Testing	13	13	0
Integration Testing	13	13	0
Total	81	80	1

Overall pass rate: 80 of 81 cases passed, approximately 98.77%. The single failing case is an authentic bcrypt 72-byte input limitation that was identified during unit testing and mitigated by enforcing a 64-character maximum password length at the sign-up form layer.

Work Division:

Asna Hammad (FA23-BDS-051):

- All the test cases for Web Dashboard (Whole 1.1)

Awais Ashraf (FA23-BDS-043)

- All the test cases for Mobile App (Whole 1.2)